

Secure Authentication and Encryption With Distributed Management for SAGIN via Signcryption and Sharding Blockchain

Yizhong Liu¹, Member, IEEE, Boyu Zhao¹, Xuqi Huang, Runhua Xu¹, Member, IEEE, Xuejun Zhang¹, Member, IEEE, Jianwei Liu¹, Senior Member, IEEE, Qianhong Wu¹, Member, IEEE, Willy Susilo², Fellow, IEEE, and Robert H. Deng³, Fellow, IEEE

Abstract—With the development of air transportation, Space-Air-Ground Integrated Network (SAGIN) are playing an increasingly important role in optimizing air traffic management and enhancing flight safety for billions of passengers and trillions dollars of aviation industry. As the key technology of SAGIN, the Automatic Dependent Surveillance-Broadcast (ADS-B) system is widely used due to its simple operation, low construction cost, and high information accuracy. However, the security problems in ADS-B system, including lack of identity authentication between all communication links, crucial information transmitted in plaintext, and susceptibility to the single point of failure, have been serious obstacle to its wide application. Existing solutions fail to account for the unique characteristics of ADS-B and SAGIN, leading to inadequate security and poor performance in these specialized contexts. Aiming to solve the above issues and provide security and scalability for ADS-B system, we conduct the following research. Firstly, an enhanced identity-based broadcast signcryption (e-IBBSC) scheme is designed to keep crucial information confidential and all messages authenticated simultaneously. Secondly, we propose an efficient batch message authentication method combined with the Merkle tree and proposed e-IBBSC, significantly improving the ADS-B message utilization ratio from 1.35% to 74.10%. Thirdly, we utilize the sharding blockchain and Byzantine fault tolerance protocol to design the first sharding-based distributed management system for SAGIN that realizes fault tolerance and scalability. Finally, after a detailed security analysis and comprehensive performance

evaluation, we demonstrate that our solution can achieve all proposed system goals including security, scalability, and high performance of 1s flight transaction processing latency and 62KTPS throughput.

Index Terms—Sharding blockchain, signcryption, ADS-B, Merkle tree, Byzantine fault tolerance.

I. INTRODUCTION

SPACE-AIR-GROUND Integrated Network (SAGIN) integrates satellites, aerial platforms, and ground networks to provide real-time data communication. SAGIN enables global air traffic monitoring, greatly enhancing flight safety and airspace efficiency for billions of passengers and trillions dollars of aviation industry [1], [2]. In addition to managing aviation networks, SAGIN also encompasses low-altitude uncrewed aerial vehicle (UAV) networks, which have shown great promise and are rapidly evolving in areas such as information gathering, logistics, and auxiliary communications. SAGIN supports emergency response, optimizes route planning, enables global network connectivity, and provides real-time communication [3].

As one of the key technology of SAGIN, Automatic Dependent Surveillance-Broadcast (ADS-B) works on the Global Navigation Satellite System (GNSS) [4] to offer an all-weather aircraft node surveillance service [5]. As shown in Fig. 1, in a typical ADS-B, each participant retrieves its position and speed through an onboard GNSS sensor [6]. Then, each aircraft use the ADS-B data link to automatically and continuously broadcast its ID, position, speed, and more option information like flight intent or emergency code, achieving information sharing among aircraft or between aircraft and the Air Traffic Control center (ATC). ATC utilizes and analyzes massive ADS-B messages to generate a comprehensive aviation management situation.

Compared with radar surveillance technology, ADS-B data comes directly from onboard equipment. It leads the higher surveillance data accuracy and better real-time performance. ADS-B can also provide surveillance services in areas without radar coverage, significantly increasing airspace capacity. Therefore, main countries and regions around the world, such as Australia [7], European [8] and American [9], are promoting the application of the ADS-B system. China has also achieved the goal of the entire airspace ADS-B OUT monitoring, and the next target is to improve the operational network of ADS-B OUT by 2025 and begin implementing the operation plan of ADS-B IN (receiver). Other countries such as Japan, Singapore, and Canada are also responding on a large scale and actively deploying ADS-B systems.

Received 12 December 2024; revised 12 July 2025; accepted 15 December 2025; approved by IEEE TRANSACTIONS ON NETWORKING Editor N. Zhang. Date of publication 23 December 2025; date of current version 12 January 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2025ZD0808500; in part by the National Natural Science Foundation of China under Grant T2425023, Grant U21B2021, Grant U22B2008, Grant U21A20467, Grant U22A1213, Grant 62202027, Grant 62172025, Grant 62472015, and Grant 62302022; in part by Beijing Natural Science Foundation under Grant L251003; in part by Zhejiang Provincial Natural Science Foundation of China under Grant LMS25F020014; in part by the Open Research Fund of The State Key Laboratory of Blockchain and Data Security, Zhejiang University, and in part by the Fundamental Research Funds for the Central Universities, Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing. (Corresponding authors: Runhua Xu; Jianwei Liu.)

Yizhong Liu, Boyu Zhao, Xuqi Huang, Xuejun Zhang, Jianwei Liu, and Qianhong Wu are with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China (e-mail: liuyizhong@buaa.edu.cn; boyuzhao@buaa.edu.cn; huangxuqi@buaa.edu.cn; zhxj@buaa.edu.cn; liujianwei@buaa.edu.cn; qianhong.wu@buaa.edu.cn).

Runhua Xu is with the School of Computer Science and Engineering, Beihang University, Beijing 100191, China (e-mail: runhua@buaa.edu.cn).

Willy Susilo is with the School of Computing and Information Technology, Institute of Cybersecurity and Cryptology, University of Wollongong, Wollongong, NSW 2522, Australia (e-mail: wsusilo@uow.edu.au).

Robert H. Deng is with the School of Information Systems, Singapore Management University, Singapore 178902 (e-mail: robertdeng@smu.edu.sg). Digital Object Identifier 10.1109/TON.2025.3647703

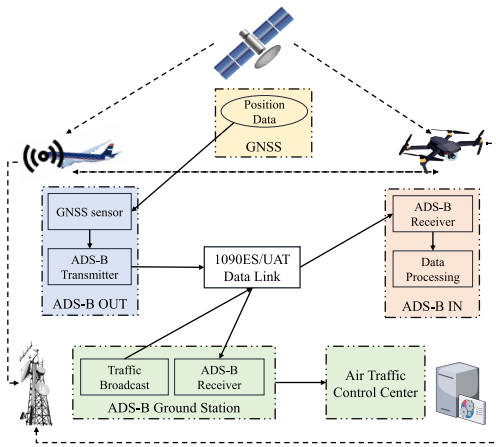


Fig. 1. A classical ADS-B system architecture for SAGIN.

Security flaws in existing ADS-B systems. However, there are multiple severe security problems in existing ADS-B systems [10]. The ADS-B messages are transmitted through an open wireless channel without security and verification measures. Therefore, attackers can intercept, modify, inject, and replay messages at will and carry out various attacks between aircraft and ATC centers [11]. Specifically, we identify the following main security issues in ADS-B systems.

- **Lack of identity authentication between ground-air and air-air.** Attackers can arbitrarily forge any participants in the ADS-B system and modify messages, causing ADS-B data packets to suffer from message integrity and identity forgery attacks [10]. Attackers can also forge false management centers and provide false monitoring and command signals to aircraft, directly affecting flight safety [12].
- **Message transmission in plaintext without information confidentiality protection.** Attackers can intercept, eavesdrop, and interfere with messages, which may cause crucial information leakage and abuse. Although some countries like the UK have enacted laws to prohibit unintended recipients from monitoring ADS-B broadcast messages [13], the technical reality makes these laws meaningless.
- **Single point of failure and low performance caused by high centralization.** As the number of air travelers and flights increase, aviation transactions also rise significantly, posing a considerable challenge to the single ATC node and amplifying the risk of a single point of failure. In August 2023, the National Air Traffic Services Company (NATS) of the UK experienced a critical single point of failure in its ATC system. This event led to over 2,000 flight cancellations, impacting an estimated 250,000 passengers. The disruption of air services persisted for up to a week [14].

Challenges. To address the above issues, traditional solutions are no longer applicable since the following challenges persist.

First, ADS-B operates in a unidirectional manner with no two-way interaction. Traditional certificate-based authentication methods which require interactive certificate exchange during authentication is unsuitable for ADS-B. Second, ADS-B messages are broadcast in open channel without fixed recipients. However, conventional point-to-point asymmetric or symmetric key encryption requires a deterministic recipient

for each encryption and ciphertext transmission. That is to say, for n recipients, it is necessary to encrypt n times and send n messages, unsuitable for the ADS-B broadcast scenario. Third, ADS-B message has short message and high transmission frequency. Each ADS-B packet length is only 112 bits, and a message is sent approximately every 0.5 seconds. Considering that numerous aircraft in airspace frequently generate ADS-B messages, the communication overhead and the computational overhead of security protection are significant. Besides, existing distributed management of ATC merely replicates servers, which fails to achieve fault tolerance or scalability, and does not meet the demands of managing massive transactions.

Our design road. To tackle the above challenges, we fully consider the characteristics of ADS-B and SAGIN, and make the following designs.

For identity authentication, we design an identity-based cryptography scheme with short signature length, short public key (i.e. identity) length, and no interaction. A sender only needs to send a shorter signature each time while a recipient is only required to retain an authorized identity list before taking off for identity verification. For message confidentiality, we consider the broadcast characteristic of ADS-B. We adopt the identity-based broadcast encryption where a sender only needs to pre-define a potential aircraft identity list along the flight. For one message and n receivers, a sender only needs to encrypt once, and broadcasts a singular message. Only the pre-defined receivers can decrypt the message. Hence, we make improvements to reduce the computation and communication overhead. If authentication and encryption are separated, both a sender and a verifier need to execute two-step operations. We combine two processes and design a *novel enhanced identity-based broadcast signcryption* algorithm to achieve message authentication and encryption in one time.

Besides, to further reduce communication overhead and computation overhead, we adopt the Merkle tree algorithm [15] to compress multiple ADS-B packets and achieve *batched verification*. ADS-B messages are seen as Merkle tree leaf nodes and the hash function is utilized to calculate the root value. All messages could realize data integrity and immutability through signing on the single root.

Moreover, the existing centralized ATC architecture is vulnerable to the threat of single-point failure. We combine the promising blockchain technology [16] to design a new ATC architecture. By replacing a centralized ATC node with a committee of ATC nodes, multiple ATC nodes jointly run *blockchain consensus* to process aviation transactions such as route information and aircraft identity management with high efficiency, strong fault tolerance, and immutability. Furthermore, considering the wide area and the massive routes managed by ATC committee, it is challenging for a single ATC committee to handle the requests. Therefore, we introduce *sharding technology* [17] to divide ATC nodes in different regions into multiple shards to improve processing scalability. Each single shard processes the transactions related to its own management area while multiple shards jointly handle the transactions related to multi-area such as international flights.

A. Contributions

In this paper, we design a secure *authentication* and *encryption* with *distributed management* (AEDM) scheme and make the following contributions.

- **The first sharding-based secure and scalable distributed management for ATC.** We propose the first

sharding-based secure and scalable distributed management architecture for ATC in SAGIN. ATC nodes run Byzantine Fault Tolerance (BFT) consensus. For a query, ATC nodes run BFT and response with high efficiency. Aircraft transactions including identity lists and crucial messages will be committed on the blockchain and get immutability. Moreover, by utilizing a region-based sharding structure, ATC nodes only need to process issues related to their region. When flights operate in multiple regions, ATC nodes can easily handle requests through cross-shard protocols. AEDM realizes security, high performance, and scalability, simultaneously.

- Enhanced identity-based broadcast signcryption scheme.** We propose a new identity-based broadcast signcryption (e-IBBSC) scheme. By introducing a virtual user identity, e-IBBSC achieves indistinguishability secure under selective identity against adaptive chosen-ciphertext attack (IND-sID-CCA2) security higher than traditional schemes' indistinguishability secure under selective identity against chosen-plaintext attack (IND-sID-CPA) security. e-IBBSC firstly achieves confidentiality and authentication for ADS-B in one step. Besides, compared with related schemes [27], [28], [29], e-IBBSC has a shorter ciphertext length and remains in the same order of computational cost. e-IBBSC not only integrates seamlessly with ADS-B systems but also demonstrates universal applicability across various broadcast communication scenarios, such as submarine wireless sensor networks and satellite ad hoc networks.
- Efficient authentication method realizing high message utilization.** We propose a high message utilization ADS-B authentication method based on e-IBBSC and Merkle tree. Continuous ADS-B messages in a period are compressed into a Merkle tree. Only the tree root and crucial ADS-B messages are signcrypt by e-IBBSC. We also propose a new formal indicator Message Utilization Ratio (MUR) to measure ADS-B message utilization ratio (ADS-B message length to the total message length). Our solution has a high MUR of 74.10% compared with 18% of related solutions.
- Security analysis and evaluation of AEDM.** We give formal security analysis of e-IBBSC and AEDM protocol where confidentiality, unforgeability, and liveness are satisfied. Additionally, we conduct detailed experiment evaluations. With 15 ADS-B receivers, the e-IBBSC signcryption and unsigncryption latency is 0.181s and 0.167s, respectively. With 30 ATC network nodes and 100Mbps bandwidth, the overall intra-shard latency of AEDM remains 1 second and the throughput can reach 11KTSPS. With 7 shards and 30 ATC network nodes in each shard, the overall throughput can be elevated to 62KTSPS.

B. Related Work

Non-cryptography solutions for ADS-B authentication. Non-encryption methods mainly judge the physical validity of the message by the position changes displayed in the message, including classical multi-point positioning methods such as distance boundary [30], [31], Kalman filtering [18], and group verification [19]. These schemes provide implicit authentication for aircraft, but the messages are still transmitted in plaintext, which poses a high risk of forgery and tampering.

Besides, the implicit authentication method cannot confirm the precise change of location.

Cryptography solutions for ADS-B messages. Some works have tried to provide integrity and authenticity of ADS-B messages via cryptography solutions [32]. TESLA [21] and related derivative schemes use identity authentication methods based on symmetric keys (such as HMAC [33]) and key chain methods to achieve message verification. However, solutions based on symmetric key require the sender and receiver to share the key in advance, making deployment difficult and not guarantee non-repudiation. Employing an identity authentication solution based on asymmetric keys enhances security robustness. Digital signatures prove to be a fitting method in ADS-B protection where only message integrity and non-repudiation were previously considered [34]. HIBS-Y [22] and HIBS-H [23] utilize signature for ADS-B systems. Both schemes involve signing a single message and verifying message batches. However, the number of signature messages required to realize the security properties of the original message far exceeds the number of original message packets.

Blockchain-related solutions for management. To address the single point of failure issues and achieve fault tolerance, some blockchain-based solutions have been introduced [35], [36], [37], [38]. In these solutions, messages are recorded on a blockchain ledger, but they also have certain problems. For instance, ATMchain [25] lacks adequate consideration for the security protection of ADS-B messages. Besides, ATMchain does not handle the scalability problem of consensus nodes. BATMAN [26] allows aircraft to interact with the ground through a handshake protocol during a flight but does not consider the one-way secure authentication between aircraft.

C. Comparison

We give a comparison between AEDM and related work in Table I. We analyze several properties of related schemes, including *fault tolerance*, *data integrity*, *non-repudiation*, *unforgeability*, *confidentiality*, *scalability*, and *MUR*. MUR measures the degree to which the message number required for security measures affects the proportion of data transmitted over a link. Compared with related schemes, AEDM provides authenticity (including integrity, non-repudiation, and unforgeability) and confidentiality through e-IBBSC, achieves 1/3 ATC nodes fault tolerance by utilizing BFT, exhibits scalability by sharding, and realizes high MUR with Merkle tree.

II. PRELIMINARIES

This section gives preliminaries including ADS-B message format, blockchain, Merkle tree, bilinear pairing, and identity-based broadcast signcryption.

A. Ads-B Message Format

International Civil Aviation Organization (ICAO) has defined the standard ADS-B 1090ES packet format which transmits on the frequency band 1090 MHz. A typical 1090ES packet [39] includes the following fields as shown in Fig. 2.

- Downlink Format (DF, 5bits).** It is used to indicate the type of ADS-B packets. DF17 means the packet is from the Mode-S transponder.
- Capability (CA, 3bits)** reports the capability of an ADS-B transmitting installation based on a Mode-S transponder.

TABLE I
COMPARISON BETWEEN OTHER RELATED WORKS

Secure Scheme	Technology	Fault Tolerance	Data Integrity	Non-Repudiation	Unforgeability	Confidentiality	Scalability	MUR
Kalman Filter [18]	NC	0	✓	✗	✗	✓	✗	-
Group Verification [19]	NC	0	✗	✗	✗	✗	✗	-
FingerPrint [20]	NC	0	✓	✓	✓	✗	✗	-
TESLA [21]	SC	0	✓	✗	✓	✗	✗	17.95%
HIBS-Y [22]	AC	0	✓	✓	✓	✗	✗	1.35%
HIBS-H [23]	AC	0	✓	✓	✓	✗	✗	1.35%
PKI+ECDSA [24]	AC	0	✓	✓	✓	✗	✗	8.86%
ATMchain [25]	BR	1/3	✓	✓	✓	✗	✗	-
BATMAN [26]	BR	1/3	✓	✓	✓	✗	✗	-
AEDM (Ours)	BR+Sharding +AC+Merkle	1/3	✓	✓	✓	✓	✓	74.10%

1) The symbol "✗" represents that the scheme does not satisfy the property, while "✓" indicates that the scheme satisfies the property.

2) In solution type, "NC" = Non-cryptography, "SC" = symmetric cryptography, "AC" = asymmetric cryptography, "BR" = blockchain-related

1090ES ADS-B Packet 112b			
DF+CA 8b	AA 24b	ME 56b	PI 24b
Control	Address	Message	Parity

Fig. 2. The structure of a 1090ES link packet.

- ICAO Address (AA, 24bits). It furnishes a distinct identifier for a aircraft by utilizing the last 24 bits of the ICAO address assigned to that particular aircraft.
- Message (ME, 56bits). It is the ADS-B service message including altitude, latitude, longitude, etc.
- Parity Information (PI, 24bits). It is a cyclic redundancy check code (CRC) to provide transmission error detection.

B. Blockchain Technology

Blockchain is the fundamental technology behind Bitcoin, introduced by Satoshi Nakamoto in 2008 [16]. It is a decentralized database or ledger that participating nodes establish and maintain together through a specific consensus algorithm in a distributed network. In a blockchain, several technologies are combined and blocks are arranged end-to-end, forming a chain-like data structure. Blockchain can provide unique features such as decentralization, tamper-proofing, transparency, traceability, and high reliability. In our scheme, BFT algorithms such as PBFT [40], BFT-SMART [41], Tendermint [42], and Hotstuff [43] are used as the blockchain consensus to achieve high throughput and low processing time. The BFT algorithms has the *safety* and *liveness* properties [44], [45], [46]. The safety property guarantees any submitted invalid transactions will be eventually rejected by all honest nodes and not recorded on the blockchain, while the liveness property guarantees any submitted valid transactions will finally recorded on the blockchain in a finite time.

Improving blockchain scalability. However, blockchain faces scalability problems, and the system performance does not improve as the number of nodes increases [47]. To enhance the blockchain scalability, *sharding blockchain* is first introduced by ELASTICO [48] via improving parallelism of consensus processes. In sharding blockchains, consensus nodes are allocated to different shards, with each shard responsible for processing and maintaining its own blockchain in parallel. Transactions processing may require the involvement of multiple shards. Typical sharding schemes include SSHC [49], Cheribum [50], and Kronos [51].

C. Merkle Tree

A Merkle tree [15] is a tree-like data storage structure used to efficiently verify the integrity of large data sets. Each leaf

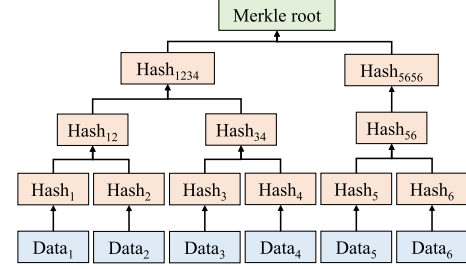


Fig. 3. The structure of a Merkle tree.

node's value is the data hash, while the values of non-leaf nodes are obtained by hashing the values of their child nodes. A Merkle tree is constructed layer by layer from bottom to top until there is only one root node left, known as the Merkle root, which can represent all the data in the tree. The utilization of a collision-resistant and one-way hash function in the construction of a Merkle tree ensures that any alteration to the data will yield a distinct Merkle root. A typical binary Merkle tree is shown in Fig. 3.

D. Bilinear Pairings

Bilinear pairs [52], a kind of binary mapping on two groups, have emerged frequently as a basic cryptography tool that is widely used in modern cryptography. Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ be three bilinear groups of prime order p . g_1 and g_2 are the generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. A bilinear pairing is a map $\hat{e}: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ with the following properties.

- Bilinearity: For all $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$ and $u, v \in \mathbb{Z}_q^*$, $\hat{e}(g_1^u, g_2^v) = \hat{e}(g_1, g_2)^{uv}$;
- Non-degeneracy: $\hat{e}(g_1, g_2) \neq 1$;
- Computability: There exists an efficient algorithm to compute $\hat{e}(g_1, g_2)$ for all $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$.

E. Identity-Based Broadcast Signcryption Scheme

Identity-Based Broadcast Signcryption (IBBSC) enables confidential information to be securely sent to authorized users through a broadcast channel while implementing user authentication to the broadcaster [53]. A typical IBBSC scheme is composed of the following algorithms.

- **Setup**(λ) $\rightarrow$ (mpk, msk): Given λ as the security parameter, the private key generator (PKG) outputs the public parameters mpk and master secret key msk .
- **KeyExtract**(mpk, msk, ID) $\rightarrow$ (sk_{ID}): Given mpk , msk , and $ID \in \{0, 1\}^*$, PKG outputs the secret key sk_{ID} .

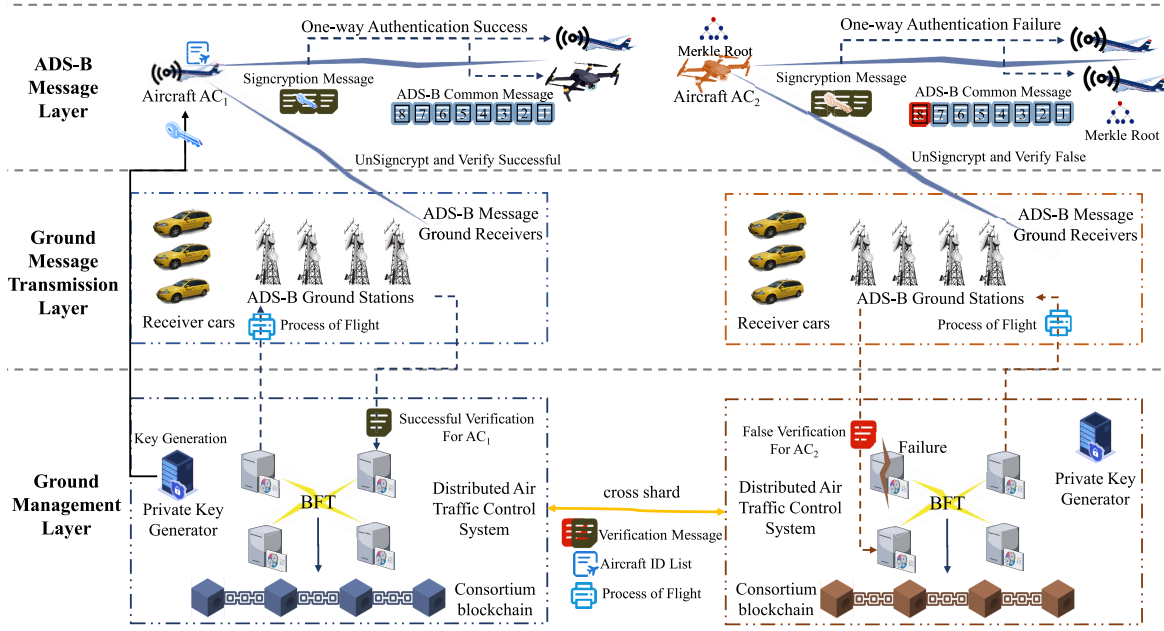


Fig. 4. An overview of our proposed AEDM system.

- $\text{Signcrypt}(mpk, \mathcal{L}, sk_{ID_s}, m) \rightarrow (\sigma)$: Given mpk , a message $m \in \{0, 1\}^*$ and an identity list $\mathcal{L} = \{ID_i\}_{i=1}^t$ of t receivers, a sender uses its secret key sk_{ID_s} to output a signed ciphertext σ .
- $\text{UnSigncrypt}(\sigma, mpk, sk_{ID_r}, ID_s) \rightarrow m/\perp$: Given σ and mpk , a receiver uses its secret key sk_{ID_r} and identity of a sender ID_s to calculate m and verify σ . If σ is valid, outputs m . Otherwise, outputs $\perp$.

III. SYSTEM OVERVIEW

In this section, we describe system models, system architecture, and system goals.

A. System Model

Adversary model. A probabilistic polynomial time (P.P.T.) adversary $\mathcal{A}$ satisfies the following limits.

- $\mathcal{A}$ cannot forge digital signatures of any honest nodes.
- $\mathcal{A}$ is a Byzantine adversary with the ability to corrupt honest nodes within the ATC committee. The number of corrupted nodes, denoted as f , satisfies $n \geq 3f+1$, where n represents the total number of ATC nodes.

Network model. An authenticated channel is adopted by each two ATC nodes. The network is partially synchronous, where messages between honest nodes are sure to arrive at each other after an unknown but limited time bound Δ .

Mathematical hard problems. $\mathcal{A}$ cannot solve the following mathematical problems.

- q -Strong Diffie-Hellman Problem (q -SDHP): g is the generator of $\mathbb{G}$, Given $g, g^\alpha, g^{\alpha^2}, \dots, g^{\alpha^q}$, the q -SDHP is to find a pair $(\beta, g^{\frac{1}{\beta+\alpha}})$ with $\beta \in \mathbb{Z}_q^*$.
- (f, g, n, F) -General Decisional Diffie-Hellman Exponent Problem ((f, g, n, F) -GDDHEP): Let $\mathcal{G} = (q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, \hat{e}(\cdot, \cdot))$ be a bilinear map, g_1 and g_2 be the generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. Let $\alpha, \beta, \gamma, t, n$ be secret random exponents in $\mathbb{Z}_q^*$, f and g be two known random polynomials defined as follows.

$$f(X) = \prod_{i=1}^t (X + x_i), g(X) = \prod_{i=t+1}^{t+n} (X + x_i)$$

Given $T \in \mathbb{G}_T$ which is random, and

$$g_1, g_1^\alpha, \dots, g_1^{\alpha^{t-1}}, g_1^{\alpha \cdot f(\gamma)}, g_1^{\beta \cdot \alpha \cdot f(\alpha)}, g_2, g_2^\alpha, \dots, g_1^{\alpha^{2n}}, g_2^{\beta \cdot g(\alpha)}$$

(f, g, n, F) -GDDHEP is to decide $T \stackrel{?}{=} \hat{e}(g_1, g_2)^{\beta \cdot f(\alpha)}$.

B. System Architecture

Fig. 4 describes the architecture of our proposed AEDM system, which contains three layers.

ADS-B message layer. In this layer, we use AC to denote an aircraft with ADS-B onboard equipment. An AC continuously broadcasts its flight status information m via the ADS-B OUT link. Common information is denoted as m_{com} and is broadcast directly, while crucial information is denoted as m_{cr} and protected by the proposed e-IBBSC scheme (in Section IV) to obtain data integrity, confidentiality, and authentication. The aircraft also receives information m from other aircraft or ADS-B ground stations via the ADS-B IN link.

Ground message transmission layer. The ground message transmission layer is essential for the data interaction between the ADS-B message and the ground management layer. This layer mainly contains ADS-B ground stations (GS) and some other receivers like ADS-B cars. The entities in this layer receive, unsigncrypt, and process ADS-B messages m from AC. After sending messages to ATC nodes, they could receive messages from ATC nodes, verify and send them to AC via the ADS-B OUT link.

Ground management layer. The management layer is composed of many shards denoted as $S^1, \dots, S^u$. Each shard contains $n = 3f + 1$ ATC nodes that run BFT to maintain a consortium blockchain $\mathcal{C}$ where ID lists, verification message, and flight information is recorded. Besides, a PKG is introduced to generate private keys for the aircraft.

If a shard receives an intra-shard request, such as when an AC completes its identity registration for the trip in a single shard, the ATC nodes in the shard run BFT algorithms to handle the request, record it on $\mathcal{C}$, and return a consistent result. The commitment certificate CC is sent to AC.

If a cross-shard request occurs, such as when AC needs to operate a cross-shard flight, it sends a request for some flight information to the ATC nodes in the registration shard. Through a cross-shard method, relative shards can handle the request, record it on C, and return a non-conflict result. CC is also generated and sent to AC.

C. System Goal

We aim to design a secure and scalable system that satisfies the following properties.

- **Security.** Security contains four aspects (informal here and formal definitions with strict proofs are given in Section VI).
 - **Confidentiality.** Any crucial message broadcast by an ADS-B sender can only be recognized by entities on a valid identity list.
 - **Unforgeability of AC.** Any message broadcast by an ADS-B sender cannot be forged by other entities.
 - **Unforgeability of ATC.** Any message generated by ATC nodes through BFT cannot be forged by other entities. Unforgeability of ATC implies consistency among ATC nodes which means honest ATC nodes must have an identical view of consensus results.
 - **System liveness.** The request submitted to the system must be processed and key information must be written on the sharding blockchain after a specific time.
- **Scalability.** Scalability is defined from two aspects.
 - **High message utilization.** The ratio of original ADS-B messages to total messages including security protection information should exceed a certain threshold.
 - **System scalability.** The entire system's ability to handle air traffic transactions can grow as the ATC nodes increase.
- **High performance.** High performance is reflected in the throughput and latency of request processing.
 - **Low latency.** The system latency can be controlled in seconds for specific scenarios.
 - **High throughput.** The system can handle massive execution requests and reaches thousands or even tens of thousands in throughput.

IV. PROPOSED E-IBBSC SCHEME

In this section, we design a new e-IBBSC protocol.

A. Proposed e-IBBSC Scheme

Our concrete e-IBBSC scheme is shown in Algorithm 1. Combined with Bohen's work [54], an extra hash function and a virtual user identity are introduced into C_3 against the adaptive chosen-ciphertext attack. Besides, we use the temporary key k to protect the authentication vector w_2 . Compared with [29], the construction decreases the collusion vulnerability caused by the authentication vector leakage and has a shorter ciphertext. More comparison and analysis are shown in Section VI.

B. Security Definitions

Next, we give security definitions of our e-IBBSC scheme. Concrete proof is given in Section VI-A.

Definition 1 (Correctness): An IBBSC scheme is correct if and only if any legal ciphertext $\sigma =$

Algorithm 1 Enhanced Identity-Based Broadcast Signcryption

► **Setup**(λ, n) $\rightarrow (mpk, msk)$ ► PKG operation

- 1: **choose** a prime $q \leftarrow \{0, 1\}^\lambda$ and a secret parameter $s \xleftarrow{\lambda} \mathbb{Z}_q^*$
- 2: **choose** groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ fulfill the following conditions:
 - $\mathbb{G}_1, \mathbb{G}_2$ with order q and basepoint g_1, g_2
 - $\mathbb{G}_T$ with order q and bilinear pairing $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$
- 3: **choose** hash functions $H_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*, H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*, H_2 : \{0, 1\}^* \times \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_T \rightarrow \mathbb{Z}_q^*, H_3 : \mathbb{G}_T \rightarrow \{0, 1\}^*, H_4 : \mathbb{G}_T \rightarrow \{0, 1\}^*$
- 4: **compute** $g = g_1^s, v := \hat{e}(g_1, g_2), \mathcal{L}_{g_2} = g_2^s, g_2^{s^2}, \dots, g_2^{s^n}$
- 5: **output** $msk = \{g_1, s\}, mpk = \{\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, q, g, v, \mathcal{L}_{g_2}, \hat{e}, \mathcal{H} = \{H_i\}\}$

► **KeyGen**(mpk, msk, ID) $\rightarrow (sk_{ID})$ ► PKG operation

- 6: **compute** $sk_{ID} := g_1^{\frac{1}{s+H_1(ID)}}$
- 7: **output** sk_{ID}

► **Signcrypt**($mpk, sk_{ID_s}, \mathcal{L}, m$) $\rightarrow (\sigma)$ ► sender operation

- 8: **choose** an one-time temporary parameter $\{\alpha, \beta\} \leftarrow \mathbb{Z}_q^*$
- 9: **compute** $K := \hat{e}(g_1, g_2)^\alpha = v^\alpha$ ► generate random key
- 10: **compute** $C_1 := g^{-\alpha}, C_2 := sk_{ID_s}^\beta$ ► mask sender's secret key
- 11: **compute** $C_3 := g_2^{\alpha(s+H_0(C_1 \cdot C_2)) \prod_{i=1}^t (s+H_1(ID_i))}$
 - bind sender and receivers
- 12: **compute** $w_1 := H_2(m, C_1, C_2, K), w_2 := H_3(v^{\beta w_1 + \alpha})$
 - generate authentication vector
- 13: **compute** $C_4 := (m \| w_2) \oplus H_4(K)$
 - encrypt message and authentication vector
- 14: **output** $\sigma = [C_1, C_2, C_3, C_4]$

► **UnSigncrypt**(σ, sk_{ID_r}) $\rightarrow m' / \perp$ ► Receiver operation

- 15: **compute** $\delta_{h,r} = H_0(C_1, C_2) \prod_{i=1, i \neq r}^t H_1(ID_i)$
- 16: **compute** $\delta_{s,r} = \frac{1}{s} \left((s + H_0(C_1, C_2)) \prod_{i=1, i \neq r}^t (s + H_1(ID_i)) - \delta_{h,r} \right)$
 - compute intermediate parameters
- 17: **compute** $K' := (\hat{e}(C_1, g_2^{\delta_{s,r}}) \cdot \hat{e}(sk_{ID_r}, C_3))^{\frac{1}{\delta_{h,r}}}$ ► recover key
- 18: **compute** $m', w'_2 := H_4(K') \oplus C_4$ ► recover plaintext
- 19: **compute** $w'_1 := H_2(m', C_1, C_2, K')$
- 20: **compute** $tmp := H_3(\hat{e}(C_2^{w'_1}, g_2^s \cdot g_2^{H_1(ID_s)}) \cdot K')$
 - recover authentication vector
- 21: **if** $tmp = w'_2$ **then** ► check the validity
- 22: **output** m'
- 23: **else**
- 24: **output** $\perp$

Signcrypt($mpk, sk_{ID_s}, \mathcal{L}, m$) can be unsigncrypt to m by UnSigncrypt($\sigma, mpk, sk_{ID_r}, ID_s$).

Definition 2 (IND-sID-CCA2 Secure): An IBBSC scheme is IND-sID-CCA2 secure if no adversary $\mathcal{A}_1$ can win the following game with a non-negligible advantage ϵ .

Init: The adversary $\mathcal{A}_1$ gives an ID set $\mathcal{L}^* = \{ID_1^*, \dots, ID_t^*\}$ to be attacked.

Setup: The challenger $\mathcal{C}$ executes the Setup algorithm for producing the public parameters mpk and master key msk , then forwards mpk to $\mathcal{A}_1$.

Phase 1: $\mathcal{A}_1$ issues the next queries.

- **Key Generation Queries:** $\mathcal{A}_1$ issues the queries with $ID \notin \mathcal{L}^*$, $\mathcal{C}$ carries out the **KeyGen** algorithm for deriving sk_{ID} and delivers it to $\mathcal{A}_1$.
- **Unsigncrypt Queries:** $\mathcal{A}_1$ issues the queries with ID and σ , $\mathcal{C}$ carries out the **Unsigncrypt** algorithm for deriving m and delivers it to $\mathcal{A}_1$.

Challenge: When $\mathcal{A}_1$ decides that phase 1 finishes and $\mathcal{A}_1$ starts to challenge, it outputs two equal-length messages m_0, m_1 . Then, $\mathcal{C}$ randomly selects $b \in \{0, 1\}$ to signcrypt m_b under $\mathcal{L}^*$ and forwards the challenge ciphertext σ^* to $\mathcal{A}_1$.

Phase 2: $\mathcal{A}_1$ issues a similar process to phase 1. The difference is that $\mathcal{A}_1$ cannot query σ^* in unsigncrypt queries.

Guess: $\mathcal{A}_1$ outputs its guess $b' \in \{0, 1\}$.

Definition 3 (EUF-CMA Secure): An IBBSC scheme is EUF-CMA secure if no adversary $\mathcal{A}_2$ can win the following game with a non-negligible advantage ϵ .

Init: The simulator $\mathcal{S}$ executes the **Setup** algorithm for producing public parameters mpk and master key msk , and forwards mpk to $\mathcal{A}_2$.

Key generation queries: $\mathcal{A}_2$ issues the queries with ID_i , $\mathcal{S}$ carries out **KeyGen** algorithm for deriving secret key and delivers it to $\mathcal{A}_2$.

Signcrypt queries: $\mathcal{A}_2$ issues the queries with m_i , sender's identity ID_s , a list of receivers' identities $\{ID_r\}_{r=1}^t$. $\mathcal{S}$ carries out the **Signcrypt** algorithm for deriving ciphertext σ_i and sends it to $\mathcal{A}_2$.

Forgery: $\mathcal{A}_2$ outputs a tuple $(\sigma^*, ID_s^*, ID_r^*)$ in which the secret key of ID_r^* is not queried. If **Unsigncrypt** $(\sigma^*, ID_s^*, ID_r^*)$ is valid, $\mathcal{A}_2$ wins.

V. PROPOSED AEDM SYSTEM

Next, we give system flows of AEDM. We divide the system processes into different phases, including system initialization, aircraft registration, flight preparation, and flight operation.

A. System Initialization

Each shard S executes the following steps to initialize as shown in Algorithm 2.

- Step 1: S.ATC generates a key pair (apk, ask) used in BFT and initializes a consortium blockchain. The key pair (apk, ask) will be used to generate the aggregate signatures as the commitment certificate (Lines 1-2).
- Step 1.1: S.PKG runs **e-IBBSC.Setup** to initialize the cryptography parameters. All S.PKG nodes share the same cryptography parameters (Line 6).
- Step 2: S.GS sends the fixed ID_{GS} to S.PKG node and S.ATC node through a trusted channel to record the S.GS (Line 11).
- Step 3: S.ATC runs BFT to record ID_{GS} (Lines 3-5).
- Step 4: S.PKG runs **e-IBBSC.KeyGen** to generate sk_{GS} , and sends it to S.GS via a trusted channel. ID_{GS} and sk_{GS} will be used to unsigncrypt the message from AC during the flight (Lines 7-10, 12-13).

After system initialization, we give two different scenarios demonstrated by Fig. 5 and Fig. 6. The term “intra-shard flight scenarios” refers to flight scenarios within a specific region or country, while “cross-shard flight scenario” indicates flight scenarios that span across regions or countries.

Algorithm 2 Initialization

-
- As an ATC member ATC of S^i
 - 1: **initialize** the blockchain C^i
 - 2: **generate** a key pair (apk, ask) for BFT and record apk on C^i
 - 3: **upon** receiving ID_{GS} from GS **do**
 - 4: **run** BFT(ID_{GS}) $\rightarrow \langle ID_{GS} \rangle_{\sigma^i}$
 - 5: **send** $\langle ID_{GS} \rangle_{\sigma^i}$ to PKG
 - As a PKG node PKG in S^i
 - 6: **run** **e-IBBSC.Setup** to initialize the cryptographic parameters
 - 7: **upon** receiving $\langle ID_{GS} \rangle_{\sigma^i}$ from ATC **do**
 - 8: **check** the validity of $\langle ID_{GS} \rangle_{\sigma^i}$
 - 9: **run** **e-IBBSC.KeyGen**(mpk, msk, ID_{GS}) $\rightarrow sk_{GS}$
 - 10: **send** sk_{GS} to GS
 - As a GS member GS of S^i
 - 11: **send** ID_{GS} to PKG and ATC
 - 12: **upon** receiving sk_{GS} from PKG **do**
 - 13: **store** sk_{GS}
-

B. Intra-Shard Flight Scenario

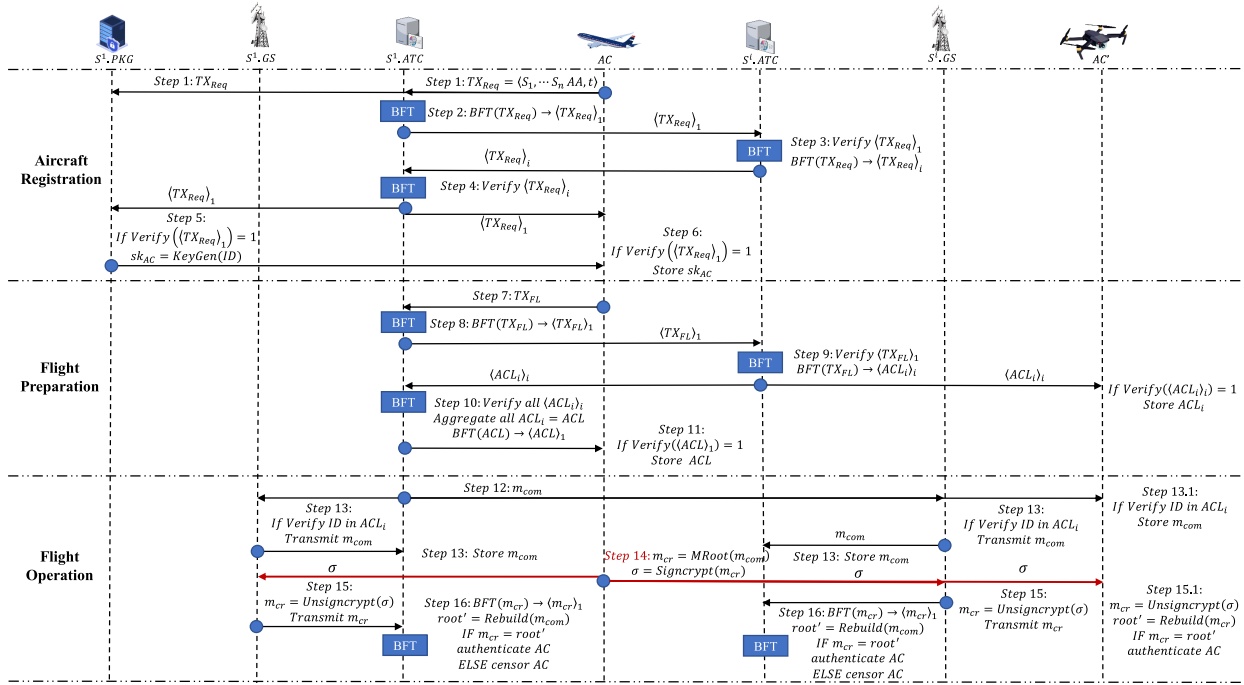
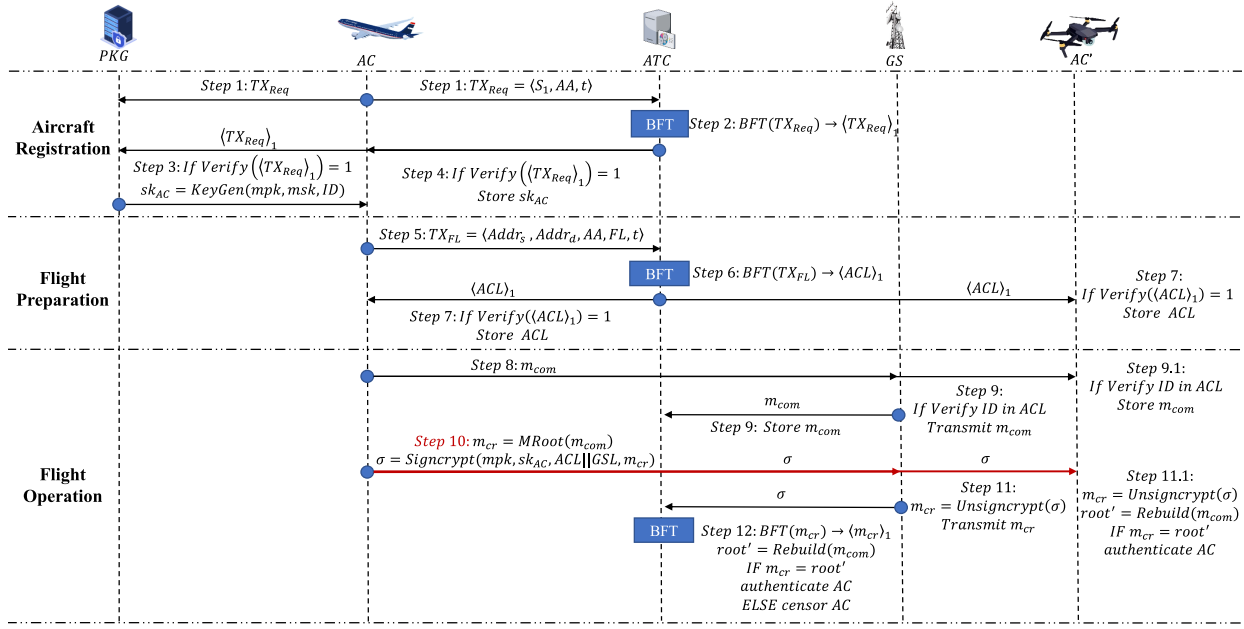
In an intra-shard flight scene (Fig. 5), the following steps need to be executed only once to complete aircraft registration (shown in Algorithm 3).

- Step 1: AC sends the registration request $TX_{Reg} = (S^1, AA, t)$ to S^1 .ATC and S^1 .PKG over a trusted channel. TX_{Reg} represents AC with ICAO address AA executes in S^1 during time t (Line 1).
- Step 2: The S^1 .ATC node runs BFT to handle the registration request TX_{Reg} and record it on the blockchain. Then the proof $\langle TX_{Reg} \rangle_{\sigma^1}$ for TX_{Reg} is sent to S^1 .PKG and AC by S^1 .ATC (Lines 10-13).
- Step 3: After receiving $\langle TX_{Reg} \rangle_{\sigma^1}$, S^1 .PKG node verifies it and calls **e-IBBSC.KeyGen** to generate private key sk_{AC} for AC. Then, the S^1 .PKG node returns sk_{AC} to AC over a trusted channel (Lines 6-9).
- Step 4: After receiving $\langle TX_{Reg} \rangle_{\sigma^1}$ from S^1 .ATC and sk_{AC} from S^1 .PKG, AC verifies $\langle TX_{Reg} \rangle_{\sigma^1}$ and stores sk_{AC} for the following signcrypt in flight operations (Lines 2-5).

After registration, before taking off, the following steps need to be executed to fulfill flight preparation (shown in Algorithm 4).

- Step 5: AC sends the flight prepare request $TX_{FL} = (Addr_s, Addr_d, AA, FL, t)$ to S^1 .ATC over a trusted channel. TX_{FL} represents AC executes flight FL with the departure $Addr_s$ and destination $Addr_d$ (Line 1).
- Step 6: S^1 .ATC nodes run BFT to handle the request TX_{FL} , decide the aircraft identity information associated with the AC as ACL , and record it on the blockchain. ACL indicates other aircraft that AC needs to one-way authenticate with each other by signcrypt during the flight. Then, $\langle ACL \rangle_{\sigma^1}$ for ACL is sent to AC by S^1 .ATC (Lines 4-7).
- Step 7: After receiving $\langle ACL \rangle_{\sigma^1}$, AC verifies it and stores ACL (Lines 2-3).

After taking off, the following steps are executed (shown in Algorithm 5).



- Step 8: During the flight, AC continuously broadcasts common ADS-B messages m_{com} to illustrate its states (Line 1).
- Step 9: When AC receives common ADS-B messages m_{com} , AC verify whether the message sender is in ACL. If so, AC provisionally stores m_{com} (Lines 6-8).
- Step 9.1: When S^i .GS receives common ADS-B messages m_{com} , it transmits m_{com} to the S^i .ATC, and then S^i .ATC store m_{com} permanently (Lines 19-21).
- Step 10: After the time interval t_{com} , AC generates the Merkle root using m_{com} during t_{com} as leaf nodes and sets Merkle root as crucial message m_{cr} . AC calls e-IBBSC.Signcrypt to compute the

signcryption message σ of m_{cr} according to ACL and the ground station list GSL . Then AC broadcasts σ . Remarkably, m_{cr} can include other crucial information (Lines 2-5).

- Step 11: When S^i .GS receives σ , it calls e-IBBSC.UnSigncrypt to calculate plaintext message. If unsigncryption is successful, S^i .GS will send m_{cr} to S^i .ATC (Lines 16-18).
- Step 11.1: When AC' receives σ , it calls e-IBBSC.UnSigncrypt to calculate the plaintext m_{cr} , rebuilds the Merkle root, and verifies that if m_{cr} equals the Merkle root. If so, AC' authenticates AC and utilizes m_{com} (Lines 9-13).

Algorithm 3 Aircraft Registration

► As an aircraft AC in S^i

- 1: **send** $TX_{\text{Reg}} = (S^1, \dots, S^n, AA, t)$ to $S^i.\text{ATC}$
- 2: **upon** receiving sk_{AC} from PKG **do**
- 3: **store** sk_{AC}
- 4: **upon** $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$ from ATC **do**
- 5: **check** the validity of $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$
- As a PKG node PKG in S^i
- 6: **upon** $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$ from ATC **do**
- 7: **check** the validity of $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$
- 8: **run** e-IBBSC.KeyGen($mpk, msk, TX_{\text{Reg}} \rightarrow sk_{\text{AC}}$)
- 9: **send** sk_{AC} to AC
- As an ATC member ATC^j of S^i
- 10: **upon** receiving TX_{Reg} from AC **do**
- 11: **run** BFT($TX_{\text{Reg}} \rightarrow \langle TX_{\text{Reg}} \rangle_{\sigma^i}$)
- 12: **if** TX_{Reg} is intra-shard **then**
- 13: **send** $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$ to AC and PKG
- 14: **else**
- 15: **send** $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$ to all relative S^j
- 16: **upon** receiving $\langle TX_{\text{Reg}} \rangle_{\sigma^j}$ from other S^j **do**
- 17: **check** the validity of $\langle TX_{\text{Reg}} \rangle_{\sigma^j}$
- 18: **run** BFT($TX_{\text{Reg}} \rightarrow \langle TX_{\text{Reg}} \rangle_{\sigma^i}$)
- 19: **send** $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$ to S^j
- 20: **upon** receiving all valid $\langle TX_{\text{Reg}} \rangle_{\sigma^j}$ from other S^j **do**
- 21: **send** $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$ to AC and PKG

Algorithm 4 Flight Preparation

► As an aircraft AC in S^i

- 1: **send** $TX_{\text{FL}} = (Addr_s, Addr_d, AA, FL, t)$ to $S^i.\text{ATC}$
- 2: **upon** receiving $\langle ACL \rangle_{\sigma^i}$ from ATC **do**
- 3: **check** the validity of $\langle ACL \rangle_{\sigma^i}$ and **store** ACL
- As an ATC member ATC of S^i
- 4: **upon** receiving TX_{FL} from AC **do**
- 5: **if** TX_{FL} is intra-shard **then**
- 6: **run** BFT($TX_{\text{FL}} \rightarrow \langle ACL \rangle_{\sigma^i}$)
- 7: **send** $\langle ACL \rangle_{\sigma^i}$ to AC
- 8: **else**
- 9: **run** BFT($TX_{\text{FL}} \rightarrow \langle TX_{\text{FL}} \rangle_{\sigma^i}$)
- 10: **send** $\langle TX_{\text{FL}} \rangle_{\sigma^i}$ to all relative S^j
- 11: **upon** receiving $\langle TX_{\text{FL}} \rangle_{\sigma^j}$ from other S^j **do**
- 12: **check** the validity of $\langle TX_{\text{FL}} \rangle_{\sigma^j}$
- 13: **run** BFT($TX_{\text{FL}} \rightarrow \langle ACL_i \rangle_{\sigma^i}$)
- 14: **send** $\langle ACL_i \rangle_{\sigma^i}$ to $S^j.\text{ATC}$
- 15: **upon** receiving all $\langle ACL_j \rangle_{\sigma^j}$ from other S^j **do**
- 16: **check** the validity of $\langle ACL_j \rangle_{\sigma^j}$
- 17: **summarize** all ACL_j into ACL .
- 18: **run** BFT($ACL \rightarrow \langle ACL \rangle_{\sigma^i}$)
- 19: **send** $\langle ACL \rangle_{\sigma^i}$ to AC

- Step 12: When $S^i.\text{ATC}$ receives m_{cr} , it runs BFT to rebuild the Merkle root from m_{com} and verify that if the m_{cr} equals the Merkle root. If so, $S^i.\text{ATC}$ authenticates AC, records m_{cr} on the blockchain, and utilizes m_{com} (Lines 22-25).

Algorithm 5 Flight Operation

► As an aircraft AC

- 1: **broadcast** common ADS-B messages and set a timer t_{com}
- 2: **upon** t_{com} is over **do**
- 3: **generate** $\text{Mroot}(m_{\text{com}}) \rightarrow m_{\text{cr}}$
- 4: **call** e-IBBSC.Signcrypt($mpk, sk_{\text{AC}}, ACL \parallel GSL, m_{\text{cr}} \rightarrow \sigma$)
- 5: **send** σ through successive ADS-B packets
- 6: **upon** receiving m_{com} from AC' **do**
- 7: **if** AC' is in ACL **then**
- 8: **store** m_{com} provisionally
- 9: **upon** receiving σ from AC' **do**
- 10: **call** e-IBBSC.UnSigncrypt($\sigma, sk_{\text{AC}} \rightarrow m_{\text{cr}}$)
- 11: **generate** $\text{Mroot}(m_{\text{com}}) \rightarrow m_{\text{cr}}'$
- 12: **if** $m_{\text{cr}}' = m_{\text{cr}}$ **then**
- 13: **authenticate** AC'
- As a GS member GS^h in S^i
- 14: **upon** receiving m_{com} from AC **do**
- 15: **transmit** m_{com} to $S^i.\text{ATC}$
- 16: **upon** receiving σ from AC **do**
- 17: **call** e-IBBSC.UnSigncrypt($\sigma, sk_{\text{GS}^h} \rightarrow m_{\text{cr}}$)
- 18: **transmit** m_{cr} to ATC
- As an ATC member ATC in S^i
- 19: **upon** receiving m_{com} of AC from GS **do**
- 20: **if** AC is in ACL **then**
- 21: **store** m_{com} permanently
- 22: **upon** receiving m_{cr} of AC from GS **do**
- 23: **generate** $\text{Mroot}(m_{\text{com}}) \rightarrow m_{\text{cr}}'$
- 24: **if** $m_{\text{cr}}' = m_{\text{cr}}$ **then**
- 25: **authenticate** AC

C. Cross-Shard Flight Scenario

In a cross-shard flight scene (Fig. 6), the following steps need to be executed only once to complete aircraft registration (shown in Algorithm 3).

- Step 1: AC sends the registration request $TX_{\text{Reg}} = (S^1, \dots, S^n, AA, t)$ to $S^1.\text{ATC}$ and $S^1.\text{PKG}$ over a trusted channel ($S^1, \dots, S^n$ denotes all shards that the flight is related as planned) (Line 1).
- Step 2: $S^1.\text{ATC}$ runs BFT to handle the registration request TX_{Reg} and record it on the blockchain. Then the proof $\langle TX_{\text{Reg}} \rangle_{\sigma^1}$ is sent to $S^2.\text{ATC}, \dots, S^n.\text{ATC}$ by $S^1.\text{ATC}$ (Lines 14-15).
- Step 3: After receiving $\langle TX_{\text{Reg}} \rangle_{\sigma^1}$, $S^i.\text{ATC}$ ($i \neq 1$) runs BFT to process TX_{Reg} and record it on the blockchain. Then the proof $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$ is sent to $S^1.\text{PKG}$ by $S^i.\text{ATC}$ (Lines 16-19).
- Step 4: After receiving all $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$ from $S^i.\text{ATC}$, $S^1.\text{ATC}$ runs BFT to verify all $\langle TX_{\text{Reg}} \rangle_{\sigma^i}$. Then the proof $\langle TX_{\text{Reg}} \rangle_{\sigma^1}$ is sent to $S^1.\text{PKG}$ and AC (Lines 20-21). This step ensures the consistency of registration.
- Step 5: After receiving $\langle TX_{\text{Reg}} \rangle_{\sigma^1}$, $S^1.\text{PKG}$ calls e-IBBSC.KeyGen to generate the private key sk_{AC} for AC over a trusted channel (Lines 6-9).
- Step 6: After receiving $\langle TX_{\text{Reg}} \rangle_{\sigma^1}$ from $S^1.\text{ATC}$ and sk_{AC} from $S^1.\text{PKG}$, AC verifies $\langle TX_{\text{Reg}} \rangle_{\sigma^1}$ and stores sk_{AC} for the following signcryption in flight operations (Lines 2-5).

After registration, before taking off, the following steps need to be executed to complete flight preparation (shown in Algorithm 4).

- Step 7: AC sends the flight prepare request $TX_{FL} = (Addr_s, Addr_d, AA, FL, t)$ to $S^1.ATC$ over a trusted channel (Line 1).
- Step 8: $S^1.ATC$ node runs BFT to handle the flight preparation request TX_{FL} and record it on the blockchain. Then the proof $\langle TX_{FL} \rangle_{\sigma^1}$ is sent to $S^2.ATC, \dots, S^n.ATC$ by $S^1.ATC$ (Lines 9-10).
- Step 9: After receiving $\langle TX_{FL} \rangle_{\sigma^1}$, $S^i.ATC$ ($i \neq 1$) runs BFT to handle the flight preparation request TX_{FL} , decides the aircraft identity information associated with the AC as ACL_i , and records it on the blockchain. ACL_i indicates other aircraft that AC needs to one-way authenticate with each other by signcryption during the flight in S^i . Then, the proof $\langle ACL_i \rangle_{\sigma^i}$ for ACL_i is sent to $S^1.ATC$ by $S^i.ATC$ ($i \neq 1$) (Lines 11-14).
- Step 10: After receiving all $\langle ACL_i \rangle_{\sigma^i}$, $S^1.ATC$ runs BFT to verify $\langle ACL_i \rangle_{\sigma^i}$, summarize ACL_i ($i = 1$ to n) to ACL , and record it on the blockchain. Then, $\langle ACL \rangle_{\sigma^1}$ is sent to AC (Lines 15-19).
- Step 11: After receiving $\langle ACL \rangle_{\sigma^1}$, AC verifies it and stores ACL . All the cross-shard flight preparations finishes (Lines 2-3).

Optimization for cross-shard flight operation. After taking off, since $S^i.PKG$ uses the same parameters, different shards can independently perform unsigncryption without the cross-shard communication. That is to say, the flight operation process (Steps 12-16 in Fig. 6) is the same as Steps 8-12 in Fig. 5. The design can ensure the high efficiency of AEDM system in high-density airspace scenarios with frequent inter-region flights. The cross-shard communication only happens in the aircraft registration and flight preparation phase and has only a limited effect on the overall system performance. At the same time, since the identity lists used in the signcryption during the flight operation are recorded on the blockchain immutably, the security is not notably compromised.

VI. SECURITY ANALYSIS

This section proves that our scheme is secure against several possible attacks and satisfies various security properties.

A. E-IBBSC Security Analysis

Theorem 1 (e-IBBSC Satisfies Correctness as Per Definition 1): Assume that ciphertext σ is the result of $e\text{-IBBSC.Signcrypt}(mpk, sk_{ID_s}, \mathcal{L}, m)$ where sk_{ID_s} and $\mathcal{L}$ are valid, then $m' = e\text{-IBBSC.Unsigncrypt}(\sigma, sk_{ID_r}) = m$.

Proof of Theorem 1: If the ciphertext σ is legal, we have:

$$\begin{aligned}
 K' &= (\hat{e}(C_1, g_2^{\delta_{sr}}) \cdot \hat{e}(sk_{ID_r}, C_3))^{\frac{1}{\delta_{hr}}} \\
 &= (\hat{e}(g_1, g_2)^{-\alpha(s+H_0(C_1, C_2))} \prod_{i=1, i \neq r}^t (s+H_1(ID_i)) \\
 &\quad * \hat{e}(g_1, g_2)^{\alpha(H_0(C_1, C_2))} \prod_{i=1, i \neq r}^t (H_1(ID_i)) \\
 &\quad * \hat{e}\left(g_1^{\frac{1}{s+H_1(ID_r)}}, g_2^{\alpha(s+H_0(C_1, C_2))} \prod_{i=1}^t (s+H_1(ID_i))\right))^{\frac{1}{\delta_{hr}}} \\
 &= \left(\hat{e}(g_1, g_2)^{\alpha(H_0(C_1, C_2))} \prod_{i=1, i \neq r}^t (H_1(ID_i))\right)^{\frac{1}{\delta_{hr}}} \\
 &= \hat{e}(g_1, g_2)^{\alpha \delta_{hr} \cdot \frac{1}{\delta_{hr}}} = K
 \end{aligned}$$

If the temporary key K is correct, other cryptographic parameters are correct obviously. $\square$

Theorem 2 (e-IBBSC Satisfies IND-sID-CCA2 Security): Assume that the (f, g, n, F) -GDDHEP holds, then e-IBBSC can achieve IND-sID-CCA2 security as per Definition 2.

Proof of Theorem 2: Assume $\mathcal{A}_1$ is able to break the e-IBBSC scheme with a non-negligible advantage ϵ . Then there will exist a challenger $\mathcal{C}_1$ that can solve (f, g, n, F) -GDDHEP through running $\mathcal{A}_1$. Given an instance:

$g_1, g_1^\alpha, \dots, g_1^{\alpha^{t-1}}, g_1^{\alpha \cdot f(\gamma)}, g_1^{\beta \cdot \alpha \cdot f(\alpha)}, g_2, g_2^\alpha, \dots, g_1^{\alpha^{2n}}, g_2^{\beta \cdot g(\alpha)}$ and $T \in \mathbb{G}_T$, the mission of $\mathcal{C}_1$ is to decide $T = \hat{e}(g_1, g_2)^{\beta \cdot f(\alpha)}$ or a random element of $\mathbb{G}_T$. n is the input of both $\mathcal{A}_1$ and $\mathcal{C}_1$, t is the total number of queries issued by $\mathcal{A}_1$. Both f and g are unitary polynomials defined as follows:

$$f(X) = \prod_{i=1}^t (X + x_i), g(X) = \prod_{i=t+1}^{t+n} (X + x_i)$$

Init: The adversary $\mathcal{A}_1$ gives an ID set $\mathcal{L}^* = \{ID_1^*, \dots, ID_{l^*}^*\}$ to be challenged, where $l^* \leq n$ holds.

Setup: The challenger $\mathcal{C}_1$ defines $g_1 = g_0^{f(s)}$ and sets

$$\begin{aligned}
 g_2 &= h_0^{\prod_{i=t+1}^{t+n} (s+x_i)}, g = g_1^s = g_0^{s \cdot f(s)} \\
 v &= \hat{e}(g_0, h_0)^{f(s) \cdot \prod_{i=t+1}^{t+n} (s+x_i)} = \hat{e}(g, h)
 \end{aligned}$$

Then $\mathcal{C}_1$ sends the public parameters $mpk = \{\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, q, g, v, g_2^s, g_2^{s^2}, \dots, g_2^{s^t}, \hat{e}, H_1, H_2, H_3, H_4\}$ to $\mathcal{A}_1$.

Phase 1: $\mathcal{C}_1$ maintains lists L_0, L_1, L_2, L_3, L_4 to simulate H_0, H_1, H_2, H_3, H_4 oracles that are initially empty.

- H_0 queries: When a query with $ID_i \notin \mathcal{L}^*$ is received, $\mathcal{C}_1$ checks if (ID_i, x_i, sk_{ID_i}) is in L_1 . If so, $\mathcal{C}_1$ returns $H_0(ID_i) = x_i$ to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_1$ randomly picks $x_i \in \mathbb{Z}_q^*$ and adds $(ID_i, x_i, *)$ into L_0 , where $*$ is described as an empty entry in L_0 . Then, $\mathcal{C}_1$ returns x_i to $\mathcal{A}_1$.
- H_1 queries: When a query with $ID_i \notin \mathcal{L}^*$ is received, $\mathcal{C}_1$ checks if (ID_i, x_i, sk_{ID_i}) is in L_1 . If so, $\mathcal{C}_1$ returns $H_1(ID_i) = x_i$ to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_1$ randomly picks $x_i \in \mathbb{Z}_q^*$ and adds $(ID_i, x_i, *)$ into L_1 , where $*$ is described as an empty entry in L_1 . Then, $\mathcal{C}_1$ returns x_i to $\mathcal{A}_1$.
- H_2 queries: When a query with $(m_i, K_i, C_{1,i}, C_{2,i})$ is received, $\mathcal{C}_1$ checks if this tuple is in L_2 . If so, $\mathcal{C}_1$ returns $H_2(m_i, K_i, C_{1,i}, C_{2,i}) = h_{2,i}$ to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_1$ randomly picks $h_{2,i} \in \mathbb{Z}_q^*$ and adds $m_i, K_i, C_{1,i}, C_{2,i}$ into L_2 . Then, $\mathcal{C}_1$ returns $h_{2,i}$ to $\mathcal{A}_1$.
- H_3 queries: When a query with (v^{r_i}) is received, $\mathcal{C}_1$ checks if v^{r_i} is in L_3 . If so, $\mathcal{C}_1$ returns $H_3(v^{r_i}) = h_{3,i}$ to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_1$ randomly picks $h_{3,i} \in \{0, 1\}^*$ and adds $(v^{r_i}, h_{3,i})$ into L_3 . Then, $\mathcal{C}_1$ returns $h_{3,i}$ to $\mathcal{A}_1$.
- H_4 queries: When a query with K_i is received, $\mathcal{C}_1$ checks if K_i is in L_4 . If so, $\mathcal{C}_1$ returns $H_4(K_i) = h_{4,i}$ to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_1$ randomly picks $h_{4,i} \in \{0, 1\}^*$ and adds $(K_i, h_{4,i})$ into L_4 .
- Key Generation queries: When a query with ID_i is received, $\mathcal{C}_1$ checks if $ID_i = ID_s^*$. If so, $\mathcal{C}_1$ aborts. Else if (ID_i, x_i, sk_{ID_i}) exists in L_1 , $\mathcal{C}_1$ returns sk_{ID_i} to $\mathcal{A}_1$. Else if $(ID_i, x_i, *)$ exists in L_1 , $\mathcal{C}_1$ calculates $sk_{ID_i} = g_0^{f_i(s)} = g_1^{\frac{1}{s+H_1(ID_i)}}$, while updating (ID_i, x_i, sk_{ID_i}) into L_1 and returns sk_{ID_i} to $\mathcal{A}_1$. Else, $\mathcal{C}_1$ makes the H_1 queries and computes sk_{ID_i} by using x_i . Afterward, $\mathcal{C}_1$ completes L_1 and forwards sk_{ID_i} to $\mathcal{A}_1$.

- Unsigncryption queries: $\mathcal{A}_1$ issues the queries with ID and σ , $\mathcal{C}_1$ first verifies the correctness by the following equation:

$$\hat{e}(g, C_3) = \hat{e}\left(C_1, g_2^{(s+H_0(C_1 \cdot C_2)) \prod_{i=1}^t (s+H_1(ID_i))}\right)$$

If it is correct, the $\mathcal{C}_1$ issues H_0 queries and calculates $\widetilde{sk}_{C_1, C_2} = g_0^{f_i(s)} = g_1^{\frac{1}{H_0(C_1 \cdot C_2)}}$. Notice that $x_i = H_0(C_1, C_2) \neq H_1(C_1, C_2)$, so $\widetilde{sk}_{C_1, C_2}$ is not the valid private key. However, the identity $x_i = H_0(C_1, C_2)$ is implicitly used in **Signcrypt**, so $\mathcal{C}_1$ can use the private key to carry on other **UnSigncrypt** algorithm steps for deriving m and deliver m to $\mathcal{A}_1$.

Challenge: When $\mathcal{A}_1$ decides that phase 1 is finished and starts to challenge, $\mathcal{C}_1$ runs the **Signcrypt** algorithm to get the challenge ciphertext $\sigma^* = (C_1, C_2, C_3, C_4)$, where:

$$\begin{aligned} C_1 &= g_0^{-\alpha \cdot s \cdot f(s)}, C_2 \in \mathbb{G}_1, C_3 = h_0^{\alpha \cdot g(s)}, \\ w_2 &= H_3(v^r) (r \in \mathbb{Z}_q^*), C_4 = (m \| w_2) \oplus H_4(K), \\ K &= T^{\prod_{i=t+l^*+1}^{t+n} x_i} \cdot \hat{e}(g_0^{\alpha \cdot s \cdot f(s)}, h_0^{\Delta_s}), \\ \Delta_s &= \frac{1}{s} \left(\prod_{i=t+l^*+1}^{t+n} (s + x_i) - \prod_{i=t+l^*+1}^{t+n} x_i \right) \end{aligned}$$

Note that if $T = \hat{e}(g_0, h_0)^{\alpha \cdot f(s)}$, we have:

$$\begin{aligned} K &= \hat{e}(g_0, h_0)^{\alpha \cdot f(s) \cdot \prod_{i=t+l^*+1}^{t+n} x_i} \cdot \hat{e}(g_0^{\alpha \cdot s \cdot f(s)}, h_0^{\Delta_s}) \\ &= \hat{e}(g_0, h_0)^{\alpha \cdot f(s) \cdot \prod_{i=t+l^*+1}^{t+n} x_i} \\ &\quad \cdot \hat{e}\left(g_0^{\alpha \cdot s \cdot f(s)}, h_0^{\frac{1}{s} (\prod_{i=t+l^*+1}^{t+n} (s+x_i) - \prod_{i=t+l^*+1}^{t+n} x_i)}\right) \\ &= \hat{e}(g_0, h_0)^{\alpha \cdot f(s) \cdot \prod_{i=t+l^*+1}^{t+n} (s+x_i)} = v^\alpha \end{aligned}$$

$\mathcal{C}_1$ randomly picks $b \in \{0, 1\}$, sets $K_b = K$ and calculates $m_b = C_4 \oplus H_4(K_b)$. Then, $\mathcal{C}_1$ defines $K_{1-b} \in \mathbb{G}_T$ and computes $m_{1-b} = C_4 \oplus H_4(K_{1-b})$. Two equal-length messages m_0, m_1 are sent to $\mathcal{A}_1$.

Phase 2: $\mathcal{A}_1$ issues a similar process to phase 1. The difference is that $\mathcal{A}_1$ cannot query σ^* in unsigncryption queries.

Guess: $\mathcal{A}_1$ outputs its guess $b' \in \{0, 1\}$ and wins the game if $b = b'$.

Following the analysis similar to [55], we can find that if $\mathcal{A}_1$ can win the game at an advantage of ϵ , $\mathcal{C}_1$ can solve (f, g, n, F) -GDDHEP at an advantage of $\frac{1}{2}\epsilon$. Assume that the (f, g, n, F) -GDDHEP holds, then $\frac{1}{2}\epsilon$ is negligible and e-IBBSC can achieve IND-sID-CCA2 security as per Definition 2. $\square$

Theorem 3 (e-IBBSC satisfies EUF-CMA security): Assume that the q -SDHP holds, then e-IBBSC can achieve EUF-CMA security as per Definition 3.

Proof of Theorem 3: Assume $\mathcal{A}_2$ is able to forge the message with a non-negligible advantage ϵ . Then there will exist a simulator $\mathcal{C}_2$ that can solve q -SDHP through running $\mathcal{A}_2$. Given an instance $(\tau, \tau^a, \tau^{a^2}, \dots, \tau^{a^q})$, $\mathcal{C}_2$ is trying to find a pair $(\kappa, \tau^{\frac{1}{a+\kappa}})$, where $\kappa \in \mathbb{Z}_q^*$. To finish the job, $\mathcal{C}_2$ executes the following steps to utilize $\mathcal{A}_2$.

Setup: $\mathcal{C}_2$ randomly picks $\theta^*, \theta_1, \theta_2, \dots, \theta_{q-1} \in \mathbb{Z}_q^*$ and expands the polynomial:

$$f(z) = \prod_{i=1}^{q-1} (z + \theta_i) = \sum_{j=0}^{q-1} r_j z^j$$

$$f_i(z) = \frac{f(z)}{z + \theta_i} + \sum_{j=0}^{q-2} \eta_j z^j$$

And then, $\mathcal{C}_2$ defines $g_1 = \tau^{f(a)}$, $g = g_1^a$ and calculates

$$W_i = \tau^{\sum_{j=0}^{q-2} \eta_j a^j} = \tau^{f_i(a)} = \tau^{\frac{f(a)}{a+\theta_i}} = g^{\frac{1}{a+\theta_i}}$$

Query phase: $\mathcal{C}_2$ maintains lists L_0, L_1, L_2, L_3, L_4 to simulate H_0, H_1, H_2, H_3, H_4 oracles that are initially empty.

- H_1 queries: When a query with ID_i is received, $\mathcal{C}_2$ checks if ID_i is in L_1 . If so, $\mathcal{C}_2$ returns $H_1(ID_i) = \theta_i$ to $\mathcal{A}_2$. Else if $ID_i = ID_s^*$, $\mathcal{C}_2$ returns $H_1(ID_i) = \theta^*$. Else, $\mathcal{C}_2$ randomly picks $\theta_i \in \mathbb{Z}_q^*$ and adds (ID_i, θ_i) into L_1 . Then, $\mathcal{C}_2$ returns θ_i to $\mathcal{A}_1$.
- H_2 queries: When a query with $(m_i, K_i, C_{1,i}, C_{2,i})$ is received, $\mathcal{C}_2$ checks if this tuple is in L_2 . If so, $\mathcal{C}_2$ returns $H_2(m_i, K_i, C_{1,i}, C_{2,i}) = h_{2,i}$ to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_1$ randomly picks $h_{2,i} \in \mathbb{Z}_q^*$ and adds $m_i, K_i, C_{1,i}, C_{2,i}$ into L_2 . Then, $\mathcal{C}_2$ returns $h_{2,i}$ to $\mathcal{A}_2$.
- H_3 queries: When a query with (v^{r_i}) is received, $\mathcal{C}_1$ checks if v^{r_i} is in L_3 . If so, $\mathcal{C}_2$ returns $H_3(v^{r_i}) = h_{3,i}$ to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_2$ randomly picks $h_{3,i} \in \{0, 1\}^*$ and adds $(v^{r_i}, h_{3,i})$ into L_3 . Then, $\mathcal{C}_2$ returns $h_{3,i}$ to $\mathcal{A}_2$.
- H_4 queries: When a query with K_i is received, $\mathcal{C}_2$ checks if K_i is in L_4 . If so, $\mathcal{C}_2$ returns $H_4(K_i) = h_{4,i}$ to $\mathcal{A}_1$. Otherwise, $\mathcal{C}_2$ randomly picks $h_{4,i} \in \{0, 1\}^*$ and adds $(K_i, h_{4,i})$ into L_4 .
- Key Generation queries: When a query with ID_i is received, $\mathcal{C}_2$ checks if $ID_i = ID_s^*$. If so, $\mathcal{C}_2$ aborts. Else if (ID_i, θ_i) exists in L_1 , $\mathcal{C}_2$ returns $W_i = g^{\frac{1}{a+\theta_i}}$ to $\mathcal{A}_2$.
- Signcryption queries: $\mathcal{A}_2$ issues the queries with m, ID and $(ID_s, \{ID_r\}_{r=1}^t)$, $\mathcal{C}_2$ checks if $ID_i = ID_s^*$. If $ID_i \neq ID_s^*$, $\mathcal{C}_2$ has knowledge of the sender's private key $sk_{ID_i} = W_i$ and runs a normal **Signcrypt** algorithm. And it needs to make some small changes for ID_j to be adaptive.

Forgery: From the forking lemma [56], $\mathcal{C}_2$ will get two signcryption messages from $\mathcal{A}_2$ through a polynomial response. To be noticed, we use the hash function H_3 to mask verification vector $w_2 = H_3(v^r)$, so we just assume that exists two signed messages $(K, C_1, C_2, C_3, C_4, w_1, m^*, w_2, v^r)$ and $(K, C_1, C_2, C_3, C_4, w'_1, m^*, w_2, v^{r'})$, in which $w_1 \neq w'_1$ and $r \neq r'$. Therefore, we have:

$$\begin{aligned} \hat{e}(C_2^{w_1}, g_2^{g_2^{H_1(ID_s^*)}}) \cdot K &= v^r \hat{e}(C_2^{w'_1}, g_2^{g_2^{H_1(ID_s^*)}}) \cdot K = v^{r'} \\ \hat{e}(C_2^{(w_1-w'_1)}, g_2^{g_2^{H_1(ID_s^*)}}) &= \hat{e}(g_1, g_2)^{(r-r')} \end{aligned}$$

Thus, $\mathcal{C}_2$ can rebuild:

$$\begin{aligned} W^* &= C_2^{(w_1-w'_1)(r-r')^{-1}} = g_1^{\frac{1}{a+\theta^*}} = \tau^{\frac{f(a)}{a+\theta^*}} \\ \frac{f(a)}{a+\theta^*} &= \frac{\eta}{a+\theta^*} + \sum_{j=0}^{q-2} \eta_j a^j \end{aligned}$$

$\mathcal{C}_2$ can find a pair $(\theta^*, \tau^{\frac{1}{a+\theta^*}})$ as the q -SDHP solution. Assume that the q -SDHP holds, then $\mathcal{C}_2$ cannot find the solutions and e-IBBSC can achieve EUF-CMA security as per Definition 3. $\square$

B. AEDM Security Analysis

Definition 4 (Confidentiality): $\forall \sigma$ sent by an ADS-B sender can only be unsigncrypted into valid m by valid entities on the authorized ACL.

Theorem 4 (AEDM Satisfies Confidentiality): Assume that $\mathcal{A}$ is restricted by system assumptions and Theorem 2 holds, then AEDM satisfies confidentiality as per Definition 4.

Proof of Theorem 4: The crucial message m_{cr} sent by AC is signcrypt into σ . By Theorem 2 and Algorithm 5, only entities in ACL and GSL can unsigncrypt σ to recover m_{cr} . $\square$

Definition 5 (Unforgeability of AC): $\forall \mathcal{A}$ cannot forge m' s.t. AC or ATC regards m' as a valid m sent by a valid ADS-B sender on the authorized ACL.

Theorem 5 (AEDM Satisfies Unforgeability of AC): Assume that $\mathcal{A}$ is restricted by system assumptions, the hash function used in Merkle tree is collision-resistant, and Theorem 3 holds, then AEDM satisfies unforgeability of AC as per Definition 5.

Proof of Theorem 5: The crucial message m_{cr} sent by aircraft is signcrypt into σ . From Theorem 3, any other entities cannot forge the σ . For common message m_{com} , due to the collision-resistant property of hash functions, any change in the leaf node m_{com} will result in a different Merkle root value m_{cr} . So all m_{com} can not be forged by any other entities. Finally, all ADS-B messages cannot be forged. $\square$

Definition 6 (Unforgeability of ATC): $\forall \mathcal{A}$ cannot forge $\langle ACL \rangle_{\sigma^i}$ s.t. AC regards σ^i as a valid BFT commitment certificate from S^i and regards ACL as valid.

Theorem 6 (AEDM Satisfies Unforgeability of ATC): Assume that $\mathcal{A}$ is restricted by system assumptions, then AEDM satisfies unforgeability of ATC as per Definition 6.

Proof of Theorem 6: Under the adversary and communication model, according to BFT algorithm, when $\mathcal{A}$ impersonates up to 1/3 ATC nodes to send incorrect messages in single shard S^i process, the honest nodes can achieve consistency of ACL and generate the signature proofs $\langle ACL \rangle_{\sigma^i}$. In the cross-shard process, $\mathcal{A}$ cannot forge the proof of each shard so $\mathcal{A}$ cannot make conflict between different shards. So ACL which ATC nodes generate can achieve unforgeability. $\square$

Definition 7 (System Liveness): A submitted request must be processed after a finite time period t .

Theorem 7 (AEDM Satisfies System Liveness): Assume that $\mathcal{A}$ is restricted by system assumptions, then AEDM satisfies system liveness as per Definition 7.

Proof of Theorem 7: We prove the system liveness property and calculate the theoretical operation time of all kinds of system behaviors.

- **Aircraft Registration Liveness.** It stipulates that a valid flight preparation transaction TX_{Reg} submitted by an AC is sure to be committed by relevant shards and written onto all honest members' blockchains. Due to the higher complexity of the cross-shard registration phase, we analyze the property in cross-shard scenarios according to Fig. 6. T_{Reg} is the time between AC sending TX_{Reg} and sk_{AC} to be sent to AC in S^1 .

- 1) Step 1 takes δ_{in} time;
- 2) By the BFT liveness property, Step 2 is sure to be carried out and takes $T_{BFT} + \delta_{cross}$ time;
- 3) By the BFT liveness property, Step 3 takes $\delta_{cross} + T_{BFT}$ time;
- 4) By the BFT liveness property, Step 4 takes $\delta_{in} + T_{BFT}$ time;
- 5) Steps 5-6 take $\delta_{keygen} + \delta_{in}$ time.

Therefore, all honest members in S^1 output a new committed block. The whole time it takes is:

$$T_{Reg} = 2\delta_{in} + 2\delta_{cross} + 3T_{BFT} + \delta_{keygen} \\ = \mathcal{O}(\delta_{in} + \delta_{cross} + T_{BFT} + \delta_{keygen})$$

Similarly, we have $T_{pr} = \mathcal{O}(\delta_{in} + T_{BFT} + \delta_{keygen})$ in the intra-shard registration phase.

- **Flight Preparation Liveness.** It denotes that a valid flight preparation transaction TX_{FL} submitted by an AC is sure to be committed by relevant shards and written onto all honest members' blockchains. Finally, AC receives the ACL from S^1 . Due to the higher complexity of the cross-shard preparation phase, we first analyze the property in the cross-shard scenarios according to Fig. 6. T_{pr} is the time between AC sending TX_{FL} and ACL to be sent to AC in S^1 .

- 1) Step 7 takes δ_{in} time;
- 2) By the BFT liveness property, Step 8 is sure to be undertaken which takes $T_{BFT} + \delta_{cross}$ time;
- 3) By the BFT liveness property, Step 9 takes $\delta_{cross} + T_{BFT}$ time;
- 4) By the BFT liveness property, Step 10 and 11 take $\delta_{in} + T_{BFT}$ time.

Therefore, all honest members in S^1 output a new committed block. The whole time it takes is:

$$T_{Pr} = 2\delta_{in} + 2\delta_{cross} + 3T_{BFT} = \mathcal{O}(\delta_{in} + \delta_{cross} + T_{BFT})$$

Similarly, we have $T_{Pr} = \mathcal{O}(\delta_{in} + T_{BFT})$ in the intra-shard preparation phase.

- **Flight Operation Liveness.** It means that an AC generates a valid m_{cr} and finally all relevant AC nodes and ATC nodes authenticate m_{cr} . In our scheme, intra-shard and cross-shard authentication are the same. So we analyze the process in Fig. 5.

- 1) Step 10 takes $\delta_{in} + \delta_{mr} + \delta_{sc}$ time;
- 2) Step 11 takes $\delta_{in} + \delta_{usc}$ time;
- 3) By the BFT liveness property, Step 12 must be completed and takes $\delta_{mr} + T_{BFT}$ time.

The whole time it takes is:

$$T_{Auth} = \mathcal{O}(\delta_{in} + \delta_{cross} + \delta_{mr} + \delta_{sc} + \delta_{usc} + T_{BFT})$$

In conclusion, under the adversary and communication model, the liveness property of BFT can hold and other steps can end in a finite time, so the system must finish all procession after a finite time t . $\square$

VII. PERFORMANCE EVALUATION

In this section, we analyze the performance of the system.

A. E-IBBSC Performance Analysis

We compare our e-IBBSC scheme with other schemes in Table II from the aspects of ciphertext length, signcrypt, and unsigncrypt calculation costs. Some notations involved are displayed in Table III for convenience.

1) *Theoretical Evaluation:* First, we carry out a theoretical analysis. As shown in Table II, we give the ciphertext length, computational costs, and security level to compare e-IBBSC and other schemes.

Although our proposal consumes slightly more signcrypt and unsigncrypt costs than the other schemes, all schemes' costs change linearly with the number of recipients t . Compared with other schemes, the extra costs of e-IBBSC are a small proportion of the overall costs.

Besides, it is obvious that our proposal has the shortest ciphertext length over all IBBSC schemes besides the original IBBS scheme. Compared with other IBBSC schemes, the ciphertext length is reduced by $|\mathbb{Z}_q^*|$ at least, which can reduce the length of transmitted data and adapt to the ADS-B transmission environment better. In security, our scheme provides

TABLE II
COMPARISONS BETWEEN OUR SCHEME AND OTHER IBBE/IBBSC SCHEMES

Scheme	Ciphertext length	Sign/Encryption costs	Unsign/Decryption costs	Cipher Security	Authvector protect
Dclerabée07 [55]	$ \mathbb{G}_1 + \mathbb{G}_2 + \mathbb{G}_T $	$T_{e1} + (t+2)T_{e2} + T_{et} + (t+1)T_{m2} + tT_h$	$2T_{bp} + (t-1)T_{e2} + T_{et} + (t-1)T_{m2} + T_{mt} + (t-1)T_h$	IND-sID-CPA	-
SVG+08 [27]	$2 \mathbb{G}_1 + \mathbb{G}_2 + m + t ID $	$2T_{e1} + (t+2)T_{e2} + T_{et} + (t+1)T_{m2} + (t+2)T_h$	$3T_{bp} + tT_{e2} + 2T_{et} + tT_{m2} + T_{mt} + (t+2)T_h$	IND-sID-CCA2	✓
KH11 [28]	$2 \mathbb{G}_1 + \mathbb{G}_2 + \mathbb{G}_T $	$2T_{e1} + (t+2)T_{e2} + T_{et} + (t+1)T_{m2} + (t+1)T_h$	$3T_{bp} + tT_{e2} + 3T_{et} + tT_{m2} + T_{mt} + (t+1)T_h$	IND-sID-CPA	✗
ZWL+23 [29]	$2 \mathbb{G}_1 + \mathbb{G}_2 + \mathbb{Z}_q^* + m $	$2T_{e1} + (t+2)T_{e2} + T_{et} + (t+1)T_{m2} + (t+2)T_h$	$3T_{bp} + T_{e1} + tT_{e2} + 2T_{et} + tT_{m2} + T_{mt} + (t+2)T_h$	IND-sID-CPA	✗
e-IBBSC (This Work)	$2 \mathbb{G}_1 + \mathbb{G}_2 + m $ (Shortest)	$2T_{e1} + (t+3)T_{e2} + 2T_{et} + (t+2)T_{m2} + (t+4)T_h$	$3T_{bp} + T_{e1} + (t+1)T_{e2} + T_{et} + (t+2)T_{m2} + T_{mt} + (t+3)T_h$	IND-sID-CCA2	✓

TABLE III
NOTATIONS

$ \mathbb{G}_1 $	A point size in $\mathbb{G}_1$	T_{e1}	Exponentiation operation in $\mathbb{G}_1$
$ \mathbb{G}_2 $	A point size in $\mathbb{G}_2$	T_{e2}	Exponentiation operation in $\mathbb{G}_2$
$ \mathbb{G}_T $	A point size in $\mathbb{G}_T$	T_{et}	Exponentiation operation in $\mathbb{G}_T$
$ \mathbb{Z}_q^* $	The bit length in $\mathbb{Z}_q^*$	T_{m1}	Multiplication operation in $\mathbb{G}_1$
$ m $	Message length	T_{m2}	Multiplication operation in $\mathbb{G}_2$
$ ID $	User's identity length	T_{mt}	Multiplication operation in $\mathbb{G}_T$
T_{bp}	Bilinear pairing operation	T_h	Hash function operation

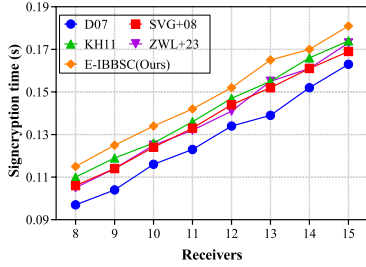


Fig. 7. Signcryption time of different receivers.

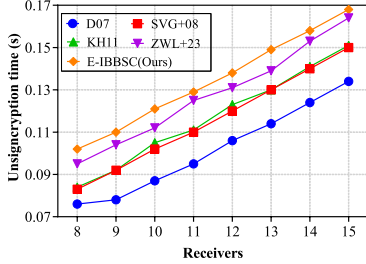


Fig. 8. Unsigncryption time of different receivers.

the highest security for ciphertext and authentication vector at the same time among all the schemes. In summary, our scheme balances security and algorithm efficiency. Besides, we adjust on ciphertext length to make the scheme suit ADS-B better.

2) *Practical Evaluation*: To evaluate the computational efficiency, we implement all schemes in Table II on a computer equipped with an Intel i5-9300H 4 core CPU working at 2.40GHz and 16GB RAM. The supersingular curve we choose is $E: y^2 = x^3 + x$ over a finite field F_q for some prime $q = 3 \bmod 4$. Both $\mathbb{G}_1$ and $\mathbb{G}_2$ are the group of points $E(\mathbb{F}_q)$, so this pairing is symmetric. The order r is some prime factor of $q+1$. The prime order of $\mathbb{G}_1$ and $\mathbb{G}_2$ is $p = 2^{159} + 2^{17} + 1$, and the prime order of $\mathbb{Z}_q^*$ is $q = 12pr - 1$. Therefore, we obtain $|\mathbb{G}_1| = |\mathbb{G}_2| = |\mathbb{G}_T| = 128$ bytes, and $|\mathbb{Z}_q^*| = 20$ bytes.

Fig. 7 and Fig. 8 depict the signcryption and unsigncryption time consumption between the five schemes with an increasing number of recipients. All results are the average of 1000 executions. It can be seen that although e-IBBSC has the most signcryption and unsigncryption time consumption, the difference between e-IBBSC and other schemes is really small with only 20ms of signcryption and 34ms of unsigncryption.

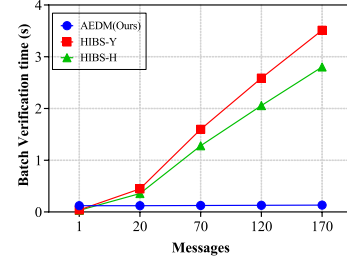


Fig. 9. Batch verification time of different messages.

B. AEDM Performance Analysis

In this section, we evaluate AEDM through simulation and illustrate that our scheme meets all scalability and performance goals mentioned in Section III-C.

1) *High Message Utilization and Authentication Efficiency*: We first give the formalization definition of the message utilization ratio.

Definition 8 (Message Utilization Ratio (MUR)):

$$MUR = \frac{l_{\text{mes}}}{l_{\text{mes}} + l_{\text{cry}}}$$

In the equation, l_{mes} stands for the message length that needs to be transmitted, and l_{cry} stands for the secure message length. The higher the MUR, the less the impact of the security measures on the link transmission.

In reality, we use SHA-256 as the hash function to construct HMAC and Merkle tree, and we assume the security message is transmitted by successive ADS-B packets. In our AEDM system, we set the authentication interval to 1 minute, and about 170 ADS-B packets are transmitted in the interval. Then we give the comparison of different cryptographic ADS-B scheme MUR under 1090ES in Table I.

Combined with Merkle tree and e-IBBSC, our AEDM can achieve crucial message confidentiality and all message authenticity at the same time with the highest MUR among all cryptographic methods. All schemes can provide authenticity, some schemes like HIBS-H and HIBS-Y even take signature batch verification into consideration. However, all other solutions ignore the fact that for short messages, it is far more effective and efficient to authenticate *batched messages* than to authenticate messages *in batches*. In addition to achieving a higher MUR, AEDM offers greater computation efficiency. As shown in Fig. 9, although HIBS-Y [22] and HIBS-H [23] could realize batch verification, the total message authentication time increases dramatically with the number of messages because they depend on computationally intensive exponentiations on elliptic curve group elements to authenticate messages *in batches*. In contrast, AEDM only uses hash function and Merkle tree to authenticate *batched messages*.

2) *Low Latency*: We simulate and evaluate the flight transaction processing latency of AEDM system under different

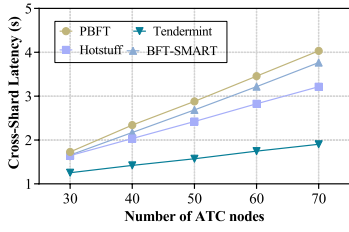


Fig. 10. Flight processing latency of different ATCs.

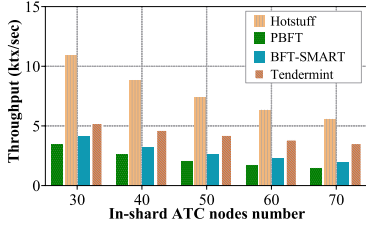


Fig. 11. Intra-shard throughput of different ATCs.

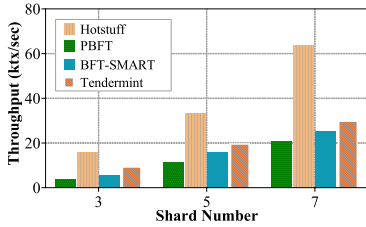


Fig. 12. Cross-shard throughput of different shards.

BFT algorithms. We set the communication bandwidth to 100Mbps and propagation delay to 10ms between ground entity nodes. All BFT algorithms propose blocks as 500KB. The signcryption and unsigncryption time is also calculated in overall time. Fig. 10 shows the AEDM system authentication latency under different baseline BFT algorithms including PBFT, Pipelined-Hotstuff, BFT-SMART, and Tendermint. The latency increases as node number rises and varies based on the consensus algorithm used. In the best case, the latency is less than 1 second.

3) *High Throughput*: We simulate and evaluate the overall process throughput of AEDM system under different BFT algorithms. We assume a transaction is 250 Bytes. Fig. 11 shows the AEDM system TPS in the intra-shard scenario under baseline BFT algorithms. Fig. 12 shows the overall AEDM system TPS under baseline BFT algorithms. We set the maximum nodes number as 210 and the shards number changes from 3 to 7.

From Fig. 11, in the intra-shard scenario, although the throughput decreases with nodes and there are differences according to the consensus algorithm used, the system performance goals in Section III-C are met in all cases, and the throughput can reach 11KTPS in the best case. From Fig. 12, in the cross-shard scenario, the total throughput gradually increases with the increase of shards. The result illustrates our system can achieve scalability through sharding technology. For 7 shards and 30 nodes per shard, the best system throughput can reach 62KTPS. Furthermore, the optimized BFT algorithm can be used as a pluggable component to improve the system throughput much more.

VIII. DISCUSSIONS

A. Comparative Analysis of AEDM and Distributed Database-Aided Schemes

Our proposed AEDM offers unique advantages over traditional distributed databases in the context of ATC. First, compared to the global trustworthy authority used in traditional distributed databases, AEDM allows multiple ATC parties to jointly participate in the consensus and management process to protect aviation interests across regions. Second, AEDM adopts BFT protocols rather than crash fault tolerance protocols (such as Raft or Paxos) used in traditional distributed databases. With the strong safety and liveness of BFT protocols, AEDM can resist unexpected errors and malicious adversaries and securely and efficiently handle aviation transactions in geographically dispersed ATC systems. Thirdly, compared to distributed databases that are vulnerable to data loss or tampering, AEDM employs blockchain to ensure permanent and immutable recording of aviation data, thereby supporting reliable traceability and auditing of flight paths. Furthermore, AEDM enhances the scalability through the sharding technology to realize the large-scale aviation management. In addition, AEDM can combine the on-chain and off-chain collaborative optimization to reduce the overhead. For example, messages can be transmitted off-chain, with only their digests recorded on the blockchain.

B. Analysis of Computation Cost

In our proposed AEDM, for k ADS-B messages transmitted within a single authentication interval t_{com} , the aircraft runs the Mroot with $\mathcal{O}(k)$ hash operations and one-time e-IBBSC.Signcrypt (Line 5 and Row 3 in Table II). As for the ground stations, it runs the Mroot with $\mathcal{O}(k)$ hash operations and one-time e-IBBSC.UnSigncrypt (Line 5 and Row 4 in Table II). In our evaluation, the signcryption and unsigncryption only need 0.181s and 0.167s, respectively. In practice, ADS-B system produces approximately 170 messages per minute. Therefore, the choice of authentication interval t_{com} introduces a trade-off between security and performance. A shorter interval leads to more frequent signcryption operations and degrades overall system performance, while a longer interval increases vulnerability to adversarial attacks, thus reducing security.

Furthermore, in AEDM, the aircraft and ground stations can perform pre-computation of time-consuming cryptographic operations to optimize system performance. For the aircraft, the intermediate parameter $Inter = g_2 \prod_{i=1}^t (s + H_1(ID_i))$ can be pre-computed because $g_2^s, \dots, g_2^{s^t}$ and $H_1(ID_1), \dots, H_1(ID_t)$ are known in advance. Consequently, in e-IBBSC.Signcrypt, the most time-consuming computation on C_3 can be significantly accelerated. For the ground station, similar pre-computation can be executed to enhance the efficiency of e-IBBSC.UnSigncrypt.

IX. CONCLUSION

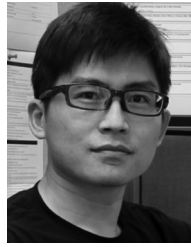
In this paper, we propose AEDM, a secure and scalable system utilizing signcryption and sharding blockchain. With the enhanced identity-based broadcast signcryption scheme design, the system addresses the security issues of the current ADS-B technology. The use of Merkle tree and sharding blockchain enhances the system scalability significantly. The proposed system is expected to revolutionize the aviation

industry by providing a secure and scalable solution over the current technology for air traffic management and SAGIN.

REFERENCES

- [1] S. Wandelt and C. Zheng, "Toward smart skies: Reviewing the state of the art and challenges for intelligent air transportation systems (IATS)," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 10, pp. 12943–12953, Oct. 2024.
- [2] J. Liu, L. Bai, C. Jiang, and W. Zhang, "Introduction to SAGIN security," in *Space-Air-Ground Integrated Network Security*. Cham, Switzerland: Springer, 2023, pp. 1–22.
- [3] W. Xu et al., "Minimizing the deployment cost of UAVs for delay-sensitive data collection in IoT networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 2, pp. 812–825, Apr. 2022.
- [4] R. Crow, "Aviation's next generation global CNS/ATM system," in *Proc. IEEE Position Location Navigat. Symp.*, Apr. 2002, pp. 291–298.
- [5] S.-H. Lee, Y.-K. Kim, J.-W. Han, and D.-G. Lee, "Protection method for data communication between ADS-B sensor and next-generation air traffic control systems," *Information*, vol. 5, no. 4, pp. 622–633, Dec. 2014.
- [6] *Minimum Aviation System Performance Standards for Automatic Dependent Surveillance Broadcast (ADS-B)*, RTCA, Incorporated, RTCA Special Committee, Washington, DC, USA, 2002.
- [7] (2017). *How ADS-B Works*. [Online]. Available: <https://www.casa.gov.au/operations-safety-and-travel/airspace/automatic-dependent-surveillance-broadcast-ads-b/how-ads-b-works#DatasentthroughADS-B>
- [8] *Commission Implementing Regulation (Eu), no. 1207/2011*, Europe, European Commission, Brussels, Belgium, 2011.
- [9] T. Smith, "Automatic dependent surveillance broadcast (ADS-B) out performance requirements to support air traffic control (ATC) service final rules," *FAA Rule*, vol. 14, pp. 1–4, Jan. 2010.
- [10] Z. Wu, T. Shang, and A. Guo, "Security issues in automatic dependent surveillance-broadcast (ADS-B): A survey," *IEEE Access*, vol. 8, pp. 122147–122167, 2020.
- [11] M. Riahi Manesh and N. Kaabouch, "Analysis of vulnerabilities, attacks, countermeasures and overall risk of the automatic dependent surveillance-broadcast (ADS-B) system," *Int. J. Crit. Infrastruct. Protection*, vol. 19, pp. 16–31, Dec. 2017.
- [12] B. Prince. (2012). *Air Traffic Control Systems Vulnerabilities Could Make for Unfriendly Skies [Black Hat]*. [Online]. Available: <https://www.securityweek.com/air-traffic-control-systems-vulnerabilities-could-make-unfriendly-skies-black-hat/>
- [13] (2012). *Civil Aviation Act 2012*. [Online]. Available: <https://www.icao.int/safety/airnavigation/AIG/Database1Docs/U.K.%20Civil%20Aviation%20Act%202012.pdf>
- [14] G. Topham. (2023). *UK Air Traffic Failure Blamed on 'Extremely Rare' Circumstances As CAA Opens Inquiry*. [Online]. Available: <https://www.theguardian.com/business/2023/sep/06/uk-air-traffic-failure-blamed-on-extremely-rare-circumstances-as-caa-opens-inquiry>
- [15] R. Merkle, "One way hash functions and DES," in *Advances in Cryptology—CRYPTO 89 Proceedings*, vol. 435. New York, NY, USA: Springer-Verlag, 1989, pp. 428–446.
- [16] S. Nakamoto. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [17] M. Zamani, M. Movahedi, and M. Raykova, "RapidChain: Scaling blockchain via full sharding," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 931–948.
- [18] Y. Hwan Chang, Q. Hu, and C. J. Tomlin, "Secure estimation based Kalman filter for cyber-physical systems against adversarial attacks," 2015, *arXiv:1512.03853*.
- [19] K. Sampigethaya, R. Poovendran, S. Shetty, T. Davis, and C. Royalty, "Future E-enabled aircraft communications and security: The next 20 years and beyond," *Proc. IEEE*, vol. 99, no. 11, pp. 2040–2055, Nov. 2011.
- [20] H. Zha, Q. Tian, and Y. Lin, "Real-world ADS-B signal recognition based on radio frequency fingerprinting," in *Proc. IEEE 28th Int. Conf. Netw. Protocols (ICNP)*, Oct. 2020, pp. 1–6.
- [21] A. Perrig, D. Song, R. Canetti, J. D. Tygar, and B. Briscoe, *Timed Efficient Stream Loss-tolerant Authentication (TESLA): Multicast Source Authentication Transform Introduction*, document RFC 4082, 2005.
- [22] A. Yang, X. Tan, J. Baek, and D. S. Wong, "A new ADS-B authentication framework based on efficient hierarchical identity-based signature with batch verification," *IEEE Trans. Services Comput.*, vol. 10, no. 2, pp. 165–175, Mar. 2017.
- [23] D. He, N. Kumar, K. R. Choo, and W. Wu, "Efficient hierarchical identity-based signature with batch verification for automatic dependent surveillance-broadcast system," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 2, pp. 454–464, Feb. 2017.
- [24] W.-J. Pan, Z.-L. Feng, and Y. Wang, "ADS-B data authentication based on ECC and X.509 certificate," *J. Electron. Sci. Technol.*, vol. 10, no. 1, pp. 51–55, 2012.
- [25] X. Lu and Z. Wu, "ATMChain: Blockchain-based security framework for cyber-physics system in air traffic management," *Secur. Commun. Netw.*, vol. 2022, pp. 1–11, Mar. 2022.
- [26] A. Arora and S. K. Yadav, "BATMAN: Blockchain-based aircraft transmission mobile ad hoc network," in *Proc. ICCCN. Cham, Switzerland: Springer*, 2018, pp. 233–240.
- [27] S. S. D. Selvi, S. S. Vivek, R. Gopalakrishnan, N. N. Karuturi, and C. P. Rangan, "Cryptanalysis of Mu et al.'s and Li et al.'s schemes and a provably secure ID-based broadcast Signcryption (IBBSC) scheme," in *Proc. WISA*, vol. 5379. Cham, Switzerland: Springer, 2008, pp. 115–129.
- [28] I. T. Kim and S. O. Hwang, "An efficient identity-based broadcast signcryption scheme for wireless sensor networks," in *Proc. Int. Symp. Wireless Pervasive Comput.*, Feb. 2011, pp. 1–6.
- [29] Y. Zhao, Y. Wang, Y. Liang, H. Yu, and Y. Ren, "Identity-based broadcast signcryption scheme for vehicular platoon communication," *IEEE Trans. Ind. Informat.*, vol. 19, no. 6, pp. 7814–7824, Jun. 2023.
- [30] S. Brands and D. Chaum, "Distance-bounding protocols (extended abstract)," in *Proc. EUROCRYPT*, 2009, pp. 344–359.
- [31] Z. P. Languell and Q. Gu, "Securing ADS-B with multi-point distance-bounding for UAV collision avoidance," in *Proc. IEEE 16th Int. Conf. Mobile Ad Hoc Sensor Syst. (MASS)*, Nov. 2019, pp. 145–153.
- [32] C. Yao, X. Zhang, Y. Liu, B. Zhao, P. Tiwari, and N. Kumar, "A blockchain-enabled secure and decentralized ADS-B system for intelligent vehicles with robust authentication," *IEEE Trans. Veh. Technol.*, vol. 74, no. 7, pp. 11294–11309, Jul. 2025.
- [33] J. J. Haas, Y.-C. Hu, and K. P. Laberteaux, "Real-world VANET security protocol performance," in *Proc. GLOBECOM IEEE Global Telecommun. Conf.*, Nov. 2009, pp. 1–7.
- [34] J. Baek, Y.-J. Byon, E. Hableel, and M. Al-Qutayri, "An authentication framework for automatic dependent surveillance-broadcast based on online/offline identity-based signature," in *Proc. 8th Int. Conf. P2P, Parallel, Grid, Cloud Internet Comput.*, Oct. 2013, pp. 358–363.
- [35] H. A. Damis, D. Shehada, C. Fachkha, A. Gawanmeh, and J. N. Al-Karaki, "A microservices architecture for ADS-B data security using blockchain," in *Proc. 3rd Int. Conf. Signal Process. Inf. Secur. (ICSPIS)*, Nov. 2020, pp. 1–4.
- [36] R. Reisman, "Air traffic management blockchain infrastructure for security, authentication, and privacy," in *Proc. AIAA Scitech Forum*, 2019, pp. 1–14.
- [37] Y. Liu, X. Huang, B. Zhao, J. Lu, and X. Zhang, "A blockchain-based secure ADS-B system," in *Proc. KSEM*, 2024, pp. 230–242.
- [38] C. Yao, X. Zhang, Y. Liu, B. Zhao, Q. Wu, and W. Susilo, "Blockchain-based secure and efficient ADS-B authentication via certificateless signature with packet loss tolerance," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 10574–10588, Apr. 2025.
- [39] *Minimum Operational Performance Standards for Universal Access Transceiver (UAT) Automatic Dependent Surveillance Broadcast (ADS-B)*, document RTCA (Firm). SC-186, RTCA, 2004.
- [40] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. 3rd USENIX Symp. Operating Syst. Design Implement.*, 1999, pp. 173–186.
- [41] A. Bessani, J. Sousa, and E. E. P. Alchieri, "State machine replication for the masses with BFT-SMART," in *Proc. 44th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Jun. 2014, pp. 355–362.
- [42] Y. Amoussou-Guenou, A. D. Pozzo, M. Potop-Butucaru, and S. Tucci-Piergiovanni, "Correctness and fairness of tendermint-core blockchains," *IACR Cryptol. ePrint Arch.*, p. 574, Jan. 2018.
- [43] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "HotStuff: BFT consensus with linearity and responsiveness," in *Proc. ACM Symp. Princ. Distrib. Comput.*, Jul. 2019, pp. 347–356.
- [44] X. Liu, Z. Zheng, W. Wang, H. Xu, F. Xiao, and K. Li, "Towards cost-effective and robust packaging in multi-leader BFT blockchain systems," *IEEE Trans. Comput.*, vol. 73, no. 11, pp. 2590–2604, Nov. 2024.
- [45] J. He et al., "An efficient multiparty payment protocol for IoT micropayments," *IEEE Internet Things J.*, vol. 11, no. 20, pp. 32854–32867, Oct. 2024.

- [46] H. Xu, X. Liu, C. Zhang, W. Wang, J. Wang, and K. Li, "Crackle: A fast sector-based BFT consensus with sublinear communication complexity," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2024, pp. 1–10.
- [47] Y. Liu et al., "Building blocks of sharding blockchain systems: Concepts, approaches, and open problems," *Comput. Sci. Rev.*, vol. 46, Nov. 2022, Art. no. 100513.
- [48] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2016, pp. 17–30.
- [49] Y. Liu, J. Liu, Q. Wu, H. Yu, Y. Hei, and Z. Zhou, "SSHC: A secure and scalable hybrid consensus protocol for sharding blockchains with a formal security framework," *IEEE Trans. Depend. Secure Comput.*, vol. 19, no. 3, pp. 2070–2088, May 2022.
- [50] A. Liu et al., "CHERUBIM: A secure and highly parallel cross-shard consensus using quadruple pipelined two-phase commit for sharding blockchains," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 3178–3193, 2024.
- [51] Y. Liu et al., "Kronos: A secure and generic sharding blockchain consensus with optimized overhead," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2025, pp. 1–18.
- [52] V. S. Miller, "The Weil pairing, and its efficient calculation," *J. Cryptol.*, vol. 17, no. 4, pp. 235–261, Sep. 2004.
- [53] F. Li, X. Xin, and Y. Hu, "Identity-based broadcast signcryption," *Comput. Standards Interfaces*, vol. 30, nos. 1–2, pp. 89–94, Jan. 2008.
- [54] D. Boneh, C. Gentry, and B. Waters, "Collusion resistant broadcast encryption with short ciphertexts and private keys," in *Proc. 25th Annu. Conf. Adv. Cryptol. (CRYPTO)*, vol. 3621. Cham, Switzerland: Springer, 2005, pp. 258–275.
- [55] C. Delerablée, "Identity-based broadcast encryption with constant size ciphertexts and private keys," in *Proc. 13th Int. Conf. Theory Appl. Cryptol. Inf. Secur. Adv. Cryptol.*, 2007, pp. 200–215.
- [56] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *J. Cryptol.*, vol. 13, no. 3, pp. 361–396, Jun. 2000.



Runhua Xu (Member, IEEE) received the B.E. degree from Beihang University and the M.S. degree from Northwestern Polytechnic University. He is currently a Professor with Beihang University, where he conducts research on privacy-enhancing techniques, privacy-preserving machine learning, applied cryptography, and blockchain. He was previously a Research Staff Member with IBM Research and worked on IBM federated learning projects.



Xuejun Zhang (Member, IEEE) received the B.S. and Ph.D. degrees from the School of Electronic and Information Engineering, Beihang University, in 1994 and 2000, respectively. He is currently a Professor with the School of Electronic and Information Engineering, Beihang University. His main research interests include air traffic management, data communication, and air surveillance.



Jianwei Liu (Senior Member, IEEE) received the B.S. and M.S. degrees in electronic and information from Shandong University, China, in 1985 and 1988, respectively, and the Ph.D. degree in communication and electronic system from Xidian University, China, in 1998. He is currently a Professor with the School of Cyber Science and Technology, Beihang University, Beijing, China. His current research interests include wireless communication networks, cryptography, and information security.



ENCE AND ENGINEERING. His research interests include cryptography and blockchain.

Yizhong Liu (Member, IEEE) received the B.S. degree in electronic and information engineering and the Ph.D. degree from Beihang University, Beijing, China, in 2014 and 2021, respectively. He is currently an Assistant Professor with the School of Cyber Science and Technology, Beihang University, Beijing. He has authored over 20 publications, including IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and IEEE TRANSACTIONS ON NETWORK SCI-



Qianhong Wu (Member, IEEE) received the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2004. He is currently an Associate Research Fellow with the University of Wollongong, Wollongong, NSW, Australia. He is a Professor with the School of Cyber Science and Technology, Beihang University, Beijing, China. He has authored over 200 publications and served on the program committees of several international conferences on information security and privacy. His research interests include cryptography and information security and privacy.



Boyu Zhao received the B.S. degree in information security from Beihang University, Beijing, China, in 2023. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology, Beihang University, China. His research interests include cryptography and blockchain consensus.



Willy Susilo (Fellow, IEEE) received the master's and Ph.D. degrees from the University of Wollongong, NSW, Australia, in 1996 and 2001, respectively. He is currently a Professor and the Head of the School of Computing and Information Technology, University of Wollongong, where he is also the Director of the Centre for Computer and Information Security Research. His main research interests include cloud security, cryptography, and information security.



Xuqi Huang received the B.S. degree in information security from Beihang University, Beijing, China, in 2023. She is currently pursuing the master's degree with the School of Cyber Science and Technology, Beihang University, China. Her research interests include cryptography and blockchain consensus.



Robert H. Deng (Fellow, IEEE) received the Ph.D. degree from Illinois Institute of Technology, Chicago, IL, USA, in 1985. He is currently an AXA Chair Professor of cyber security and the Director of the Secure Mobile Center, School of Computing and Information Systems, Singapore Management University, Singapore. His research interests include applied cryptography, data security and privacy, and network security.